

Manual de referencia de spinpy

Verificación de las métricas morfométricas y de los algoritmos de optimización
de AppFinal_v2

Carlos Gonzalez-Torres

Universidad de Valparaíso

spinpy — núcleo voxel-primero para hueso trabecular

Port a Python de la mitad numérica de AppFinal_V2.m

Documento generado automáticamente a partir de los docstrings del código
fuente. No se edita a mano: se regenera con
`python docs/generar_manual.py`

Licencia MIT. La interfaz gráfica importa PyQt5 (GPL v3 o comercial);
el núcleo spinpy no depende de Qt. Ver LICENSE.

Este manual documenta el software, no valida los resultados obtenidos con él.

La verificación numérica contra soluciones manufacturadas está en
`validar_analitico.py` y en el anexo de validación del proyecto.

Indice

El paquete	4
Generacion: el campo aleatorio gaussiano	4
Funciones	5
Lectura de volúmenes de interés	7
Funciones	8
Morfometría: BV/TV, superficie, MIL, Conn.D, SMI	8
Funciones	9
Espesor local por esferas inscritas	13
Funciones	14
Función de error entre VOI y candidato	14
Funciones	15
Ajuste: búsqueda escalonada y desempate mecánico	17
Funciones	18
Homogeneización elástica periódica	21
Funciones	22
Ensayo de compresión, von Mises y Pistoia	24
Funciones	25
Ensayos FEM con motores internos (hex8 y TET10)	28
Funciones	29
Clase `ErrorMalla`	32
Motores FEM: registro, detección y proceso hijo	32
Funciones	33
Clase `Cancelado`	33
Motores FEM: problema común y resolvedores	33
Funciones	34
Clase `Cronometro`	35
Clase `NoDisponible`	35
Clase `ErrorMotor`	35
Motor NGSolve	35
Funciones	35
Motor FEniCSx	35
Funciones	36
Motor scikit-fem	36
Funciones	36
Motor SfePy	36
Funciones	37

Motor de la app	37
Funciones	37
Mallas solidas: hexaedrica y TET10	37
Funciones	38
Exportadores: Abaqus, APDL, VTU, STL	39
Funciones	39
Lote de VOIs y dispersion del generador	41
Funciones	41
Varianza, ICC, N efectivo y equivalencia TOST	43
Funciones	43
Linea de comandos	44
Funciones	44
Interfaz grafica	44
Funciones	47
Clase `Trabajador`	48
Clase `DialogoPilaTiff`	49
Clase `DialogoRecorte`	49
Clase `DialogoFigura`	49
Clase `DialogoEspesor`	49
Clase `DialogoConvergencia`	50
Clase `DialogoDistribucionFE`	50
Clase `DialogoDispersion`	50
Clase `DialogoVarianza`	50
Clase `DialogoSimulacion`	51
Clase `Deslizador`	51
Clase `DialogoBienvenida`	51
Clase `DialogoInformeAuto`	51
Clase `Visor`	52

Ajuste de microestructuras espinodales a volúmenes de hueso trabecular obtenidos por microtomografía.

Version V2.0.0.

EN PALABRAS SENCILLAS

Este manual está **generado del código**. Cada apartado reproduce la documentación que acompaña a la función en el fuente, y no una reescritura: en este proyecto los docstrings llevan las decisiones de diseño, los sesgos medidos y las advertencias sobre que números no son citables. Si algo aquí está desactualizado, lo está también en el código, que es donde hay que corregirlo.

El paquete

`__init__.py`

spinpy — núcleo voxel-primero del pipeline de spinodoides, portado a Python.

Cubre la parte de AppFinal_V2.m que va del generador a la morfometría:

generar_mascara(...) -> BW (nx, ny, nz) -> morfometria(BW, spacing)

Lee VOIs en VTK legacy (.vtk) y en .mat de MATLAB.

Ajusta un spinodoide a un VOI con la búsqueda escalonada de fitSpinodoidToVOI (etapas A/B/C, correcciones G1/G2/K2) mediante ajustar_spinodoide.

Homogeneiza elásticamente sobre la rejilla de voxels (homogeneizar).

Mide las curvaturas principales de la interfaz hueso-vacio (curvaturas_de_campo, perfil_curvaturas), que es lo único que distingue una red conectada de un montón de islas con el mismo BV/TV.

Ensayo a compresión en X, Y o Z y estima la carga de fallo con el criterio de Pistoia, devolviendo los campos de deformación efectiva y de von Mises (ensayo_compresion, ensayo_compresion_eje, ensayo_triaxial, criterio_pistoia).

Exporta el ensayo de compresión a FEBio 4 (`escribe.escribir_febio`), validado contra FEBio en `comparativa_febio/`.

Lo que NO cubre (sigue solo en MATLAB): los otros cuatro optimizadores (Pareto, bayesiano, MOBO y el best-fit rápido con término mecánico).

Generación: el campo aleatorio gaussiano

`grf.py`

grf.py — Generador de spinodoides VOXEL-PRIMERO.

Port a Python del campo aleatorio gaussiano (GRF) de Kumar et al. (2020), tal como lo implementa `spinodoid.m` de GIBBON, con las dos convenciones de muestreo de direcciones de onda que coexisten en la literatura y en el código disponible en este proyecto.

POR QUE VOXEL-PRIMERO

`AppFinal_V2.m` genera la malla triangular con `spinodoid()`, la escala al tamaño físico del VOI y después la voxeliza (`localVoxelizeMesh`) para medirla con `localMorphometryFromVoxels`. El repositorio del profesor (TPMS-Scaffolds-generator) va aún más lejos: malla tetraédrica con CGAL y métricas sobre la malla.

Para un bucle de ajuste ese camino es innecesariamente caro: mallar en cada evaluacion domina el coste. El campo GRF ya esta definido sobre una rejilla, y la mascara booleana sale de un umbral. Aqui se corta ahi: rejilla -> umbral -> mascara. La morfometria (morphometry.py) opera sobre la mascara, que es exactamente la representacion que la app usa para comparar VOI y candidato.

LAS DOS CONVENCIONES DE MUESTREO

Los thetas son semiangulos de cono alrededor de los ejes x, y, z. Un vector de onda es admisible si cae dentro de alguno de los conos activos. Como repartir numWaves vectores entre los conos NO esta fijado por la teoria, y las dos implementaciones disponibles resuelven distinto:

'rechazo' (GIBBON / AppFinal_V2, spinodoid.m:178-206) Se muestrea un candidato isotropo en la esfera y se acepta si cae en CUALQUIER cono. El reparto entre conos resulta proporcional al angulo solido de cada uno: un cono ancho recibe muchas mas ondas que uno estrecho.

'equitativo' (TPMS-Scaffolds-generator, src/TPMS.py:223) numWaves // n_conos_activos ondas por cono, cada una muestreada uniformemente dentro de su propio cono. El reparto es 50/50 aunque los conos tengan angulos muy distintos.

Coinciden solo si todos los conos activos tienen el mismo angulo (o hay uno solo, o el caso es isotropo). Con thetas=(15,45,0) el rechazo manda ~90% de las ondas al cono de 45 grados y el equitativo un 50%. Producen anisotropias DISTINTAS para los mismos thetas nominales; ver validar_analitico.py.

Ninguna es "la correcta": son dos definiciones de la misma familia. Lo que no se puede es cruzar resultados entre ambas sin declarar cual se uso.

Convencion de ejes: la mascara devuelta tiene dimension 1 = X, igual que readVTKVOI en AppFinal_V2.m y que voxelize_to_mask en el repositorio del profesor.

Funciones

```
level_set(rho, erfinv='exacta')
```

Umbral phi0 tal que $P(\text{GRF} \leq \phi_0) = \rho$ para un GRF $N(0,1)$.

$\phi_0 = \sqrt{2} * \text{erf}^{-1}(2*\rho - 1)$ [Kumar et al. 2020]

Es identico en GIBBON (spinodoid.m:264) y en el repositorio del profesor (TPMS.py:272, salvo por la aproximacion de erfinv).

```
wave_directions(num_waves, thetas, R=None, esquema='rechazo', rng=None)
```

Direcciones de onda unitarias, (num_waves, 3).

esquema : 'rechazo' (GIBBON/AppFinal_V2) | 'equitativo' (profesor)

```
region_degenerada(thetas, esquema='rechazo', num_waves=None)
```

¿Dan estos thetas EXACTAMENTE la misma estructura que (90, 90, 90)?

POR QUE PASA

Con muestreo por rechazo se aceptan, uniformes, los candidatos que caen en la UNION de los conos. Si esa union cubre toda la esfera, se aceptan todos los candidatos: el generador aleatorio se consume igual que en el caso isotropo y la realizacion es la misma bit a bit. Una region entera del espacio nominal es un solo punto, con gradiente nulo para cualquier optimizador.

CUANDO LA UNION CUBRE LA ESFERA

Una direccion unitaria $\mathbf{d}$ escapa del cono $\mathbf{i}$ si $|\mathbf{d}_i| \leq \cos(\theta_i)$. Hay un conjunto de medida positiva que escapa de los tres si y solo si existe $\mathbf{d}$ con $\mathbf{d}_i^2 < \cos^2(\theta_i)$ para todo i y $\sum \mathbf{d}_i^2 = 1$, es decir si todos los cosenos son positivos y $\sum \cos^2(\theta_i) > 1$. Por tanto:

degenerado $\Leftrightarrow$ algun $\theta_i \geq 90$ o $\sum \cos^2(\theta_i) \leq 1$

La condicion que se habia escrito —los tres por encima de 54.74 grados, el angulo de (1,1,1) a un eje— es solo el caso simetrico ($3 \cos^2 54.74 = 1$). La general es mas ancha: (70, 70, 30) tambien es isotropo, con $\sum \cos^2 = 0.98$. El bloque 17 de `tests/` lo comprueba generando.

Con muestreo EQUITATIVO no ocurre: cada cono recibe su tercio de ondas concentradas junto a su eje, cubra o no la union la esfera. Tampoco con todos los θ_i a 0 en equitativo, que ya es el caso isotropo por definicion y no una degeneracion.

CERCA DEL BORDE TAMBIEN

Fuera de la region, pero cerca, la parte de la esfera que la union NO cubre es diminuta, y si ninguno de los candidatos cae en ella la realizacion vuelve a ser exactamente la isotropa. Medido: (60, 60, 40), con $\sum \cos^2 = 1.087$, dio a 32^3 y 300 ondas la misma mascara que (90, 90, 90). Por eso se da tambien `fraccion_sin_cubrir` (sobre 20 000 puntos fijos de Fibonacci, resolucion 5e-5) y, si se pasa `num_waves`, `prob_realizacion_isotropia` = $(1 - \text{fraccion})^{\text{num_waves}}$: la probabilidad de que ninguna onda aprovechable caiga fuera de la union. Por encima de 0.5 el gradiente respecto a estos angulos es, en la practica, nulo.

Devuelve datos, no frases: `{"degenerado", "motivo", "suma_cos2", "holgura", "fraccion_sin_cubrir", "prob_realizacion_isotropia"}`. `motivo` es "" | "theta_90" | "suma_cos2" | "todos_cero" | "equitativo". `holgura` = $\sum \cos^2 - 1$: negativa o cero dentro de la region.

```
canonicalizar_thetas(thetas, esquema='rechazo')
```

Los θ_i que representan la estructura: (90, 90, 90) si son degenerados, ellos mismos si no. Sirve de clave para no evaluar dos veces un mismo punto con nombres distintos.

```
euler_R(rx_deg, ry_deg, rz_deg)
```

Matriz de rotacion $R = R_z R_y R_x$, la convencion de `generateSpinodoid`.

Replica `AppFinal_V2.m:1283-1289`. Los deslizadores de rotacion de la app solo cubren ± 90 grados y no pueden representar todas las orientaciones de un VOI; por eso existe el mecanismo `pendingR` (correccion F4), que deja a los optimizadores forzar una R arbitraria. Aqui R se acepta directamente.

```
campo_grf(resolution, wave_number, num_waves, thetas, R=None, esquema='rechazo', seed=None, domain_size=1.0, motor='numpy', precision='f64', bloque=None)
```

Evalua el GRF sobre la rejilla del dominio.

Replica `spinodoid.m:212-232`: `rejilla = linspace(0, domainSize, resolution)` en los tres ejes, `ndgrid -> dimension 1 = X` $\text{GRF}(\mathbf{x}) = \sum_i \sqrt{2/N} \cos(\text{waveNumber} \cdot \mathbf{x}_i + \gamma_i)$

Las fases se muestrean uniformes en $[0, 2\pi)$. GIBBON usa `rand_angle`, que pese a su construccion rebuscada es uniforme en $[0, 2\pi)$ — se comprobo analiticamente — de modo que `rng.uniform` es equivalente en distribucion.

El campo se evalua por bloques de PUNTOS con un producto matricial (ver `_evaluar_campo`), no por bloques de ondas con difusion. Materializar (res^3 , `num_waves`) de una vez ocuparia ~ 1.5 GB a $64^3 \times 700$.

`precision` : 'f64' (por defecto, exacta) | 'f32' ($\sim 3\times$, ver `_evaluar_campo`)

El orden de consumo del generador aleatorio —primero las direcciones, despues las fases— se mantiene identico al de la version anterior, de modo que una misma semilla sigue dando la misma realizacion.

Devuelve (GRF, dirs, fases) con GRF de forma (res, res, res), $\text{dim } 1 = X$.

```
generar_mascara(resolution, wave_number, num_waves, thetas, rho, R=None, esquema='rechazo',
seed=None, domain_size=1.0, erfinv='exacta', mayor_componente=False, motor='numpy',
precision='f64')
```

Mascara booleana solida (True = hueso), forma (res, res, res), dim 1 = X.

Fase solida = {GRF <= phi0}, la misma convencion que `spinodoid.m` usa al llamar a `isosurface(..., levelset)` con `isocaps 'enclose', 'below'`.

AVISO sobre el codigo del profesor: alli `implicit_func` devuelve GRF + phi0 y la fase solida sale de `pgal.Difference(cuboid, tpms)`, es decir de DOS inversiones de signo encadenadas. Si se toma su `implicit_func` y se umbraliza directamente en $\phi < 0$ se obtiene la fase PORO, no la osea. Aqui la convencion es explicita y directa.

`mayor_componente` : si True, conserva solo la mayor componente conexa por caras (conectividad 6), analogo a `localLargestComponent6`. La app filtra la mayor componente de SUPERFICIE via `tesgroup`, que no es exactamente lo mismo; por eso el valor por defecto es False y el filtro se declara explicitamente cuando se quiere.

```
derivadas_grf(pts, dirs, fases, wave_number, num_waves=None, bloque=None)
```

Gradiente y hessiano ANALITICOS del GRF en una nube de puntos.

El campo es una suma de cosenos, asi que sus derivadas son exactas y no hace falta diferenciarlo numericamente:

$$\phi(x) = \sqrt{2/N} \sum_q \cos(\beta \langle n_q, x \rangle + \gamma_q) \quad \text{grad } \phi = -\sqrt{2/N} \beta \sum_q n_q \sin(\dots) \quad \text{hess } \phi = -\sqrt{2/N} \beta^2 \sum_q n_q n_q(x) \cos(\dots)$$

PARA QUE SIRVE. La curvatura de la isosuperficie sale de estas dos cantidades (`curvatura.curvaturas_implicitas`), de modo que un spinodoide tiene curvatura EXACTA, sin error de mallado ni de diferencias finitas. Es la respuesta cerrada contra la que se valida el estimador discreto que luego hay que aplicar al hueso real, donde no hay campo analitico.

`dirs` y `fases` son los que devuelve `campo_grf`; la misma semilla da las mismas y por tanto el mismo campo. `num_waves` se toma de `dirs` si no se da: el factor $\sqrt{2/N}$ depende de N y ponerlo a mano invita a que no coincida con el campo del que salieron las direcciones.

Devuelve (grad, hess) con formas (n, 3) y (n, 3, 3).

`_6` funcion(es) interna(s) de este modulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz publica._

Lectura de volúmenes de interes

`io.py`

`io.py` — Lectura de VOIs en VTK legacy ASCII (STRUCTURED_POINTS).

Port de `readVTKVOI` (AppFinal_V2.m:5147-5205) y de la binarizacion que aplica `computeVOIMetrics` (AppFinal_V2.m:5229).

CONVENCION DE EJES — la invariante que hay que respetar

El formato VTK escribe el indice X como el que varia mas rapido, asi que `reshape(data, [nx ny nz])` de MATLAB (orden por columnas) deja la DIMENSION 1 = X. Es lo que documenta la correccion del bug de ejes X/Y intercambiados en `computeVOIMetrics`: la version anterior asignaba la dimension 1 a Y y con voxeles anisotropos —lo normal en micro-CT— deformaba la geometria medida y devolvía una direccion principal erronea, que es la que fija la rotacion aplicada al spinodoide.

En numpy el equivalente exacto de ese reshape es `order='F'`. Usar el orden C por defecto transpone el volumen en silencio.

Funciones

```
leer_vtk_voi(ruta, binarizar=True, umbral=0)
```

Lee un VOI de un VTK legacy ASCII STRUCTURED_POINTS.

Devuelve (VOI, spacing) con VOI de forma (nx, ny, nz), dimension 1 = X.

binarizar : aplica $\text{VOI} > \text{umbral}$, igual que `computeVOIMetrics`, que hace `if ~islogical(VOI), VOI = VOI > 0; end`. Los VOIs de H4 vienen como `unsigned_char` con valores 0 y 255.

```
leer_mat_voi(ruta, binarizar=True, umbral=0)
```

Lee un VOI guardado como `.mat` de MATLAB (variables `VOI` y `spacing`).

Es el formato del subconjunto porcino publico (Koria, Mengoni & Brockett 2020, Research Data Leeds, CC BY 4.0).

El array ya viene con dimension 1 = X porque MATLAB lo guardo asi; `scipy.io.loadmat` lo devuelve con la misma disposicion, sin necesidad de transponer.

```
leer_voi(ruta, **kw)
```

Lee un VOI eligiendo el lector por lo que sea `ruta`.

`.vtk` / `.mat` volumen ya extraido, con su `spacing` dentro del archivo. carpeta pila de rebanadas TIFF del micro-CT. `.tif/.tiff` TIFF multipagina.

Los dos ultimos casos van a `voi.leer_pila_tiff`, que ademas de apilar resuelve la ESCALA: la saca del `*_rec.log` de SkyScan o de un `tam_voxel` explicito. Un `.vtk` trae el `spacing` dentro; una pila de imagenes no trae nada, y suponer 1 mm/voxel escalaria en silencio toda la morfometria.

Devuelve siempre (`BW`, `spacing`) en mm. `leer_pila_tiff` devuelve ademas un tercer valor con la trazabilidad (umbral aplicado, de donde salio la escala); si se necesita, hay que llamarla directamente.

`_1` funcion(es) interna(s) de este modulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz publica._

Morfometria: BV/TV, superficie, MIL, Conn.D, SMI

```
morphometry.py
```

`morphometry.py` — Port de la morfometria de `AppFinal_V2.m`.

Traduccion linea a linea de: `localMorphometryFromVoxels` (`AppFinal_V2.m:5890-5996`) `localMILTensor` (`AppFinal_V2.m:5733-5866`) `localLoadBearingFraction` (`AppFinal_V2.m:6012-6030`) `localLargestComponent6` (`AppFinal_V2.m:6502-6572`)

Se respetan las invariantes documentadas del proyecto, que son correcciones ya pagadas en el codigo MATLAB y que un port ingenuo reintroduce en silencio:

- $Tb.Th = 2 \cdot BV/BS$ (modelo de placas de Parfitt, como `CTAn/Scanco`). NO $4 \cdot BV/BS$, que es el modelo de barras y duplica el valor.
- DA sale del TENSOR MIL (Harrigan & Mann 1984), nunca de la covarianza de la nube de puntos. El MIL tiene un suelo de ruido de $DA \sim 1.07$: por debajo de esa diferencia no hay significado.
- Las estructuras laminares degeneran el ajuste del elipsoide; se acota el autovalor y se marca el resultado como COTA INFERIOR (`MIL_clamped`).
- La superficie se mide con marching cubes sobre la mascara binaria, igual que el VOI real, para que el sesgo del metodo se cancele al restar.

CONVENCION DE EJES — el punto donde mas facil es equivocarse

MATLAB llama `isosurface(permute(BW,[2 1 3]), 0.5)` porque `isosurface` devuelve los vertices en convencion meshgrid (columna, fila, pagina); la permutacion previa deja $V(:,1) = X$. Es el bug B1/B2 del historial del proyecto.

En Python NO hace falta permutar: `skimage.measure.marching_cubes` devuelve los vertices en orden de indice del array, asi que con BW de forma (nx, ny, nz) los vertices ya salen como (X, Y, Z). Se usa `method='lorensen'` (marching cubes clasico) por ser el mas proximo al de MATLAB; el 'lewiner' por defecto resuelve las ambigüedades de otra manera y da areas ligeramente distintas.

Funciones

`mayor_componente_6(BW)`

Mayor componente conexas por caras. Port de `localLargestComponent6`.

`fraccion_portante(BW)`

Fraccion de material en componentes que unen las dos caras en Z.

Port de `localLoadBearingFraction (K3)`. Vale 1 cuando todo el material forma parte de caminos que atraviesan la probeta y baja al aparecer islas y munones ciegos. Se mide, no se optimiza (peso 0 en el error).

`area_superficie(BW, spacing, metodo='lorensen')`

Area de la isosuperficie 0.5 de la mascara, en unidades fisicas.

Equivale a las lineas 5942-5950 de `AppFinal_V2.m`. Devuelve 0.0 si no hay interfaz (mascara vacia o llena).

`area_superficie_partes(BW, spacing, metodo='lorensen')`

Separa el area en INTERNA (interfaz hueso-vacio) y EXTERNA (las tapas).

POR QUE HACEN FALTA LAS DOS, Y POR QUE NO SON LA MISMA COSA

La superficie que se mide sobre un VOI tiene dos partes de naturaleza distinta y no se pueden sumar sin pensar:

INTERNA la interfaz real hueso-vacio, que es donde vive el hueso: donde se remodela, donde se adhieren las celulas, donde ocurre todo. Es la que entra en BS/BV y en Tb.Th, y la unica que tiene sentido biologico. Es lo que este modulo llamaba BS a secas, y lo sigue llamando: no se cambia el nombre de una metrica que ya esta en informes y en el error del ajuste.

EXTERNA las seis caras por donde el cubo CORTA las trabeculas. No es interfaz de nada: es la huella del bisturi. Si se sumara a la interna, BS/BV subiria al reducir el VOI —mas corte por unidad de volumen— y Tb.Th bajaria, sin que la estructura hubiera cambiado en absoluto.

Esa es exactamente la invariante del proyecto ("el area excluye las seis tapas"). Lo que anade esta funcion es poder MEDIR la que se excluye, en vez de solo descartarla: su tamaño dice cuanto del VOI es borde, que es un diagnostico de si la ventana es bastante grande para lo que se mide.

COMO SE SEPARAN, CON UNA SOLA DEFINICION DE AREA

Tentador y equivocado: contar voxeles de hueso en las seis caras y multiplicar por el area del voxel. Eso mezcla dos definiciones de area —una de marching cubes y otra de conteo de cuadraditos— que difieren en torno al 8.5 % (la correccion C4 de este proyecto), y la resta de dos definiciones distintas no significa nada.

Aqui se usa la MISMA marching cubes dos veces: sobre la mascara tal cual, que deja la superficie abierta en el borde y da la interna; y sobre la mascara rodeada de una capa de vacio, que obliga al algoritmo a cerrar las tapas y da el total. La externa es la diferencia, y las dos salen de la misma regla.

Devuelve (interna, externa, total), en unidades de `spacing` al cuadrado.

```
tamano_poro(BW, spacing, n_radios=None, campo=False)
```

Tamano de poro Po.Dm: espesor local de la fase PORO, no de la osea.

Es el metodo de Hildebrand y Rueggsegger aplicado al complemento de la mascara, que es lo que usan CTAn y Scanco para Tb.Sp y lo que la literatura de andamios llama distribucion de tamano de poro.

SOBRE "HACERLO RAPIDO CON LA TRANSFORMADA DE DISTANCIA"

La transformada de distancia ya esta dentro: es el primer paso del metodo. Lo que NO vale es quedarse ahi. La distancia al hueso mas proximo, $d(x)$, no es el tamano del poro en x : vale el radio del poro solo en el eje medial y cae a cero al acercarse a la pared. Un poro cilindrico de diametro D daria valores entre 0 y $D/2$ segun donde se mire, y su media saldria del orden de $D/3$ en vez de D . El paso que falta -y que cuesta las iteraciones de `espesor.espesor_local`- es quedarse con la mayor esfera que CONTIENE el punto, no con la centrada en el. Ver la cabecera de `espesor.py`, donde esto esta medido.

QUE SE DEVUELVE, Y POR QUE UNA DISTRIBUCION Y NO UN NUMERO

Tb.Sp, tal como lo calcula morfometria, sale del modelo de placas de Parfitt: $Tb.Th * (1/BV.TV - 1)$. Es una formula, no una medida: supone que la estructura son placas paralelas. Po.Dm se MIDE sobre la geometria, y en hueso trabecular los dos no tienen por que coincidir; la diferencia entre ambos es en si misma un indicador de cuanto se aparta la estructura del modelo de placas.

Devuelve un diccionario con Po.Dm media, mediana, desviacion, percentiles y el numero de voxeles de poro medidos. Con `campo=True` en la llamada a `espesor.espesor_local` se puede recuperar el campo entero si hace falta pintarlo; aqui solo se resumen las cifras.

EL AVISO DE BORDE, Y POR QUE NO ES EL OBVIO

La transformada de distancia mide la distancia al HUESO, y fuera del array no hay hueso: por tanto trata el exterior como si el poro siguiera. Para un poro que de verdad continua fuera del VOI eso es lo correcto; para uno que se cerraba justo despues, Po.Dm sale de mas.

La primera version de este aviso contaba los voxeles de poro cuya COMPONENTE toca una cara del cubo. No sirve: en cualquier estructura de celda abierta -o sea, en todo hueso trabecular- el poro es una sola componente que toca las seis caras, y el aviso salia 100 % siempre. Medido sobre el VOI proximal de H4: 100 %, sin decir nada de nadie.

Lo que se cuenta ahora es la fraccion de poro cuya ESFERA INSCRITA se sale de la ventana, es decir los puntos cuya distancia al borde es menor que su radio: son exactamente aquellos cuyo espesor se apoya en la extrapolacion. Y ademas se dan las cifras restringidas a los que NO se salen (`PoDm_interior`): si las dos versiones se parecen, la ventana no manda; si difieren, el VOI es pequeno para los poros que contiene y hay que decirlo.

Sigue siendo un AVISO y no una correccion: no hay forma honesta de saber cuanto seguia el poro fuera del VOI.

```
niveles_cuantiles(n=N_CUANTILES)
```

Niveles $(k + 0.5) / n$: posiciones de Hazen, la convencion de `prctile` de MATLAB que usa todo el proyecto.

```
cuantiles(valores, pesos=None, n=N_CUANTILES)
```

Cuantiles de una distribucion en los niveles de `niveles_cuantiles`.

Con `pesos` (por ejemplo el area de cada vertice en una curvatura) cada valor cuenta en proporcion a su peso, con la misma posicion de Hazen: el centro de su tramo de peso acumulado. Devuelve un array, o None si no hay ningun valor utilizable.

```
curvatura_de_mascara(BW, spacing, sigma=SIGMA_CURVATURA, margen=2)
```

Curvaturas principales de una mascara BINARIA, por el camino del VOI.

Binarizar -> suavizar con una gaussiana de `sigma` voxeles -> mallar el nivel 0.5 -> Rusinkiewicz con normales tomadas del campo suavizado. Es el unico camino disponible para un VOI de micro-CT y por eso se usa tambien en el candidato, aunque del spinodoide se conozca el campo exacto: medir uno por el campo y otro por la mascara compararia dos definiciones.

Los vertices a menos de `margen` voxeles de las caras se descartan (area cero): alli la malla esta cortada y la curvatura es del recorte.

Devuelve (`k1`, `k2`, `areas`), en 1/mm si `spacing` va en mm.

```
metricas_forma(BW, spacing, tbth=False, poro=False, curvatura=False, ef=False,
n_cuantiles=N_CUANTILES)
```

Metricas de forma que puede usar la funcion objetivo, a demanda.

Cada una tiene su compuerta porque su coste es muy distinto (medido):

`tbth` espesor local del solido: `TbTh_local`, `TbTh_CV` y cuantiles. Una transformada de distancia por radio. `poro` tamaño de poro `Po.Dm` (`tamaño_poro`) y sus cuantiles. 0.7 s a 64^3 , 12 s a 128^3 . `curvatura` `H_desv`, `H_medio`, fracciones concava/silla y cuantiles de `H`, ponderados por area. Una gaussiana y una marching cubes. `ef` Ellipsoid Factor (`elipsoide.factor_elipsoide`). 373 s a 48^3 : NUNCA dentro de un bucle de ajuste; el ajuste lo deja para los finalistas.

Las cifras de espesor local NO son citables como `Tb.Th` (sesgo de ~30 % a 3-4 voxeles, ver `espesor.py`); sirven para comparar VOI y candidato medidos con el mismo voxel, que es lo que hace el ajuste.

```
tensor_mil(BW, spacing, n_dirs=61, n_lines=32, min_chord_frac=0.35, muestreo='voxel',
sigma_vox=0.7)
```

Mean Intercept Length y tensor de fabrica. Port de localMILTensor.

MUESTREO (correccion M1, OPCIONAL; por omision no cambia nada) "voxel" lo de siempre: cada muestra de la recta lee el voxel que la contiene. Identico bit a bit a localMILTensor de MATLAB. "suavizado" la mascara se suaviza con una gaussiana de `sigma_vox` voxeles y cada muestra lee ese campo por interpolacion trilineal, con umbral 0.5.

Por que existe (Estudio_MIL): sobre voxeles, una recta que corre OBLICUA a lo largo de un puntal cruza una y otra vez la escalera de su superficie y cuenta intersecciones que no existen. El MIL cae sobre todo a lo largo del eje de la estructura, el DA baja y la direccion principal se tuerce hacia el eje de rejilla mas proximo. Medido en un dual-lattice girado de forma exacta (48^3 , estiramiento 2.5, cinco semillas): con "voxel" el error de direccion llega a 14.5 ± 1.9 grados y el DA va de 1.48 (alineado) a 1.13 segun el giro; con "suavizado" el error no pasa de 1.6 ± 0.9 grados y el DA se queda en 1.55-1.64. El DA de la estructura continua es ~1.93: NINGUN muestreo de voxeles lo recupera a 48^3 , asi que el DA absoluto sigue sin ser comparable entre resoluciones; lo que arregla M1 es la DEPENDENCIA DE LA ORIENTACION. VOI y candidato deben medirse con el mismo muestreo (C4).

Direcciones cuasi-uniformes en el hemisferio por espiral de Fibonacci; para cada direccion se lanza una rejilla de n_lines^2 rectas paralelas, se acumula longitud osea y numero de transicciones solido->poro, y

$$MIL(d) = longitud_osea_total / (0.5 * n_transiciones)$$

El factor 0.5 es la correccion F5: `totalInt` cuenta la entrada Y la salida de cada cuerda, es decir 2 por cuerda, y el MIL estandar (CTAn/Scanco) divide por el numero de cuerdas. Sin el factor se devuelve la mitad del valor. El DA no se ve afectado por ser un cociente, pero el MIL expuesto si.

Despues se ajusta por minimos cuadrados el elipsoide $1/MIL^2 = n' M n$ y se diagonaliza. El autovalor MINIMO corresponde al MIL MAXIMO, es decir a la direccion principal de la estructura.

Devuelve un dict con las mismas claves que el struct `fab` de MATLAB.

```
morfometria_malla(BW, spacing, suavizado=20, banda=0.05, do_mil=True)
```

Morfometria sobre la superficie SUAVIZADA Y REPARADA, no sobre voxeles.

Segundo modo de medida, que convive con `morfometria` y no la sustituye. Construye la isosuperficie cerrada (`solido.superficie_cerrada`), la suaviza con Taubin, la repara con PyMeshFix y la recorta a la caja del VOI. Sobre esa malla mide BV (volumen encerrado) y BS (area SIN las seis tapas del cubo), y de ahí las metricas de Parfitt. DA, DA2 y la fraccion portante siguen saliendo de los voxeles: son topologicas o del tensor MIL, y no tienen version de malla aquí (esa seria otra funcion).

POR QUE EXISTE — MEDIDO SOBRE UNA ESFERA DE AREA CONOCIDA marching cubes crudo (lo que mide `morfometria`) area +8.5 % cerrada + PyMeshFix sin suavizar area +8.5 % cerrada + Taubin(20) + PyMeshFix area +0.6..0.9 % volumen +0.1..0.4 % PyMeshFix por si solo no quita ningun sesgo: solo cierra y repara. Lo que elimina la escalera de los voxeles es el suavizado de Taubin, que a diferencia del laplaciano conserva el volumen. La pareja es lo que da una superficie "de hueso" y no "de cubos".

LO QUE CUESTA, Y SE DEVUELVE PARA QUE SE VEA

- En trabeculas de uno o dos voxeles PyMeshFix se come material al reparar: un spinodoide a 48^3 pierde ~ 1 % frente a la superficie cerrada cruda. `perdida_vs_mc_pct` lo mide en cada llamada.
- El volumen de la superficie cerrada cruda ya es ~ 3 % menor que el conteo de voxeles en estructuras finas: marching cubes coloca la superficie a mitad de camino entre centros solidos y vacios, y el conteo cuenta cubos enteros. Eso NO es una perdida, es una diferencia de definicion, y sobre una esfera el volumen de marching cubes es el mas proximo al exacto. `dif_BV_vs_voxel_pct` la reporta.

LA CAJA DE MEDIDA ES LA DE CENTROS DE VOXEL, MEDIO VOXEL POR DENTRO El suavizado REDONDEA las aristas donde una trabecula cortada toca la cara del cubo: la tapa retrocede hacia dentro, deja de ser plana y la punta redondeada se cuenta como interfaz. Medido en una losa, eso inflaba BS un 27 %. Devolver los vertices al plano lo arregla en la losa pero en un spinodoide colapsa triangulos en las puntas y deja la malla no-manifold. La solucion robusta es recortar la malla suavizada MEDIO VOXEL POR DENTRO, en los planos de los centros de voxel: el recortador de superficies cerradas elimina la zona redondeada y fabrica el mismo las tapas planas, sin tocar vertices. Medido: losa +0.9 % de area y +0.1 % de volumen; spinodoide $BS_{\text{malla}}/BS_{\text{voxel}} = 0.91-0.92$, que es justo el sesgo de voxel que el suavizado quita. Consecuencia: TV aquí es $((n-1)*sp)^3$, la caja de centros, que es tambien la de la superficie abierta de skimage que mide `morfometria`. Se devuelve `TV_voxel` para la referencia; BV/TV es comparable entre modos porque los dos son densidades sobre su propia caja.

LA REGLA DE ORO SE MANTIENE (correccion C4): VOI y candidato se miden con el MISMO modo. Mezclar una tabla de voxeles con una de malla es mezclar dos definiciones de BS que difieren un 9 %.

Devuelve las mismas claves que `morfometria` mas: `modo="malla"`, `BS_tapas`, `V_mc`, `V_rep`, `TV_voxel`, `BV_voxel`, `BS_voxel`, `dif_BV_vs_voxel_pct`, `perdida_vs_mc_pct`, `n_tri`, `estanca`, `bordes_abiertos` (de la malla reparada), `bordes_abiertos_recorte`, `reparado_tras_recorte`, `suavizado`, `banda`.

```
conectividad(BW, spacing)
```

Densidad de conectividad (Conn.D), Odgaard & Gundersen 1993.

$Conn = 1 - \chi$ Conn.D = $Conn / TV [1/mm^3]$

siendo χ la caracteristica de Euler de la fase solida. Cuenta las conexiones REDUNDANTES de la red: cuantas trabeculas se podrian cortar antes de partirla en dos.

POR QUE IMPORTA Y POR QUE NO LA SUSTITUYE BV/TV. En perdida osea las trabeculas no solo adelgazan: se desconectan. Dos estructuras con el mismo BV/TV y el mismo $Tb.Th$ pueden tener redes completamente distintas, y la resistencia depende de la red, no solo de cuanto material hay. Es la metrica que separa "hueso mas fino" de "hueso roto".

Se usa conectividad 26 para el solido, que es la convencion de Odgaard (emparejada con 6 para el fondo) y la que aplican CTAn y Scanco.

```
indice_smi(BW, spacing, dr_rel=0.05, metodo='lorensen')
```

Structure Model Index (Hildebrand & Ruegsegger 1997).

$$\text{SMI} = 6 \text{ BV}(\text{dBS/dr}) / \text{BS}^2$$

Mide si la estructura se parece mas a placas o a barras dilatandola un poco y viendo como cambia la superficie: en una placa el area apenas cambia ($\text{SMI} \sim 0$), en una barra crece linealmente ($\text{SMI} \sim 3$) y en una esfera crece mas deprisa ($\text{SMI} \sim 4$).

LA CRITICA, QUE HAY QUE REPORTAR JUNTO AL NUMERO. Salmon et al. (2015) mostraron que el SMI esta confundido por la CONCAVIDAD: las superficies concavas aportan area negativa al dilatar, asi que en hueso denso el indice sale negativo y deja de tener la interpretacion placa-barra. No es un artefacto de calculo, es una limitacion de la definicion. Se incluye porque se sigue pidiendo en la literatura, no porque sea fiable a BV/TV alto.

La dilatacion se hace desplazando los vertices de la isosuperficie a lo largo de sus normales, no con morfologia sobre voxeles: un paso de un voxel entero seria demasiado grosero para una derivada.

SESGO DE DISCRETIZACION, MEDIDO. Sobre geometrias de respuesta conocida:

esfera +3.45 (exacto 4) -13.8% cilindro +2.68 (exacto 3) -10.8% losa 0.00 (exacto 0) exacto

El deficit no es un fallo del calculo: marching cubes sobreestima BS un 8.9% sobre datos binarios y el SMI va como $1/\text{BS}^2$, asi que cabe esperar -15.7%. La losa sale exacta porque su dBS/dr es cero y el sesgo de BS no tiene por donde entrar. Toda implementacion sobre voxeles arrastra esto.

```
morfometria(BW, spacing, do_mil=True, modelo_tbth='plate', do_poro=False,
metodo_mc='lorensen', extra=False, do_ef=False, muestreo_mil='voxel')
```

Port de localMorphometryFromVoxels.

Es el UNICO sitio donde se calculan BV/TV, BS/BV, Tb.Th, Tb.Sp, Tb.N y DA, igual que en la app: cualquier metrica nueva debe pasar por aqui, porque el sesgo del metodo solo se cancela si VOI y candidato recorren el mismo camino (correccion C4).

spacing en mm/voxel. Devuelve un dict con las claves del struct de MATLAB.

Espesor local por esferas inscritas

espesor.py

espesor.py — Campo de espesor local (Tb.Th punto a punto).

Metodo de Hildebrand y Ruegsegger (1997), que es el que usan CTAn y Scanco: el espesor local en un punto es el DIAMETRO DE LA MAYOR ESFERA que contiene ese punto y cabe entera dentro de la estructura.

$$\text{Th}(y) = 2 * \max \{ r(x) : |y - x| \leq r(x) \}$$

siendo $r(x)$ la distancia de x al fondo, es decir el radio de la mayor esfera CENTRADA en x que cabe.

POR QUE NO BASTA LA TRANSFORMADA DE DISTANCIA

Es tentador colorear directamente por la distancia al fondo y llamarlo espesor. No lo es: $r(x)$ solo da el espesor en el eje medial, y cae a cero al acercarse a la superficie. Como la superficie es justo lo que se ve al renderizar, colorear por $r(x)$ mostraria una estructura uniformemente delgada por fuera, con el valor bueno escondido en el interior. El maximo sobre las esferas que CONTIENEN el punto —no las centradas en el— es lo que reparte el espesor del eje medial a todo el grosor de la trabecula.

SESGO DE DISCRETIZACION — MEDIDO, Y GRANDE A ESTA ESCALA

El metodo mide distancias entre CENTROS de voxel, asi que la frontera efectiva del objeto digitalizado cae medio voxel por dentro y el espesor sale corto. El sesgo depende de cuantos voxeles cruzan la estructura (cilindro, medido):

diametro (voxeles) 2 3 4 6 10 20 error 0.0% -33% -29% -5.7% -11% -5.7%

Una losa plana sale EXACTA (0.00%), porque no hay curvatura que discretizar.

Esto importa mucho aqui: los VOIs equinos tienen Tb.Th ~0.17 mm con voxel de 0.0515 mm, es decir unos 3.3 voxeles de diametro, justo en la zona donde el sesgo ronda el 30%. LOS VALORES ABSOLUTOS DE ESTE CAMPO NO SON CITABLES COMO Tb.Th.

Para lo que si sirve:

- ver DONDE la estructura es gruesa o fina, que es informacion relativa y el sesgo es sistematico;
- comparar VOI y spinodoide medidos con el MISMO voxel, porque entonces el sesgo afecta por igual a los dos y se cancela en gran medida al restar —el mismo argumento de la correccion C4.

Para un numero citable esta el Tb.Th de Parfitt que devuelve morfometria.

DIFERENCIA CON EL Tb.Th DE LA MORFOMETRIA

morfometria devuelve el Tb.Th de Parfitt, $2 \cdot BV/BS$, que es un MODELO DE PLACAS paralelas: un unico numero por VOI, obtenido de un cociente global. Este modulo da un campo, y su media NO tiene por que coincidir con aquel valor — son dos definiciones distintas, y las dos son estandar. La de Parfitt es la que reportan CTAn y Scanco por defecto y la que usa el ajuste; esta sirve para VER donde la estructura es gruesa o fina.

Funciones

`espesor_local(BW, spacing=1.0, n_radios=None)`

Campo de espesor local, en las mismas unidades que `spacing`.

Devuelve un array del tamaño de BW con el espesor en cada voxel solido y cero en el fondo.

El algoritmo recorre los radios de mayor a menor. Para cada radio r toma los centros que admiten una esfera de al menos ese radio y marca todo lo que queda a distancia $\leq r$ de alguno; lo que se marca por primera vez tiene espesor $2r$. Al ir de mayor a menor, cada voxel se queda con el mayor radio que lo cubre, que es la definicion.

`n_radios` limita cuantos niveles se prueban (uno por nivel cuesta una transformada de distancia). Con `None` se usan todos los radios distintos que aparecen, redondeados a la rejilla.

`estadisticas(esp, BW=None)`

Resumen del campo: media, mediana, desviacion y percentiles.

`muestrear_en_puntos(esp, spacing, puntos, origen=None)`

Valor del campo en coordenadas arbitrarias, por vecino mas proximo.

Los vertices de la isosuperficie caen ENTRE voxeles, y con frecuencia del lado del fondo, donde el espesor vale cero. Colorear con eso pintaria toda la piel de azul. Se dilata antes el campo con un maximo 3x3x3 para que cada vertice recoja el valor del solido contiguo.

Funcion de error entre VOI y candidato

`error.py`

`error.py` — Funcion de error morfometrico y criterios de fabrica.

Port de: `computeMorphometricError` (AppFinal_V2.m:3725-3785) `localFabricCos` (AppFinal_V2.m:6051) `localFabricAlignWeight` (AppFinal_V2.m:6076) `localTieneEje` (AppFinal_V2.m:6035)

INVARIANTE C6 — LA RAZON DE SER DE ESTA FUNCION

La version anterior de la app sumaba los terminos disponibles y saltaba EN SILENCIO los que salieran NaN. Consecuencia: un candidato en el que alguna metrica no se podia calcular acumulaba MENOS error que uno en el que si se calculaba, y por tanto parecia mejor. El optimizador quedaba sesgado justamente hacia las geometrias degeneradas, que son las que mas metricas hacen fallar.

La correccion es dividir por la suma de pesos EFECTIVAMENTE usados, de modo que el error es un promedio ponderado y no depende de cuantos terminos sobrevivan. Si se rompe esto, el sesgo vuelve sin dar ningun sintoma.

TERMINO MECANICO (J1)

La app puede anadir `Ez_rel` y `Ez_Ex`, que requieren homogeneizacion elastica. Sus pesos se multiplican por `peso_mecanico`; con 0 la funcion se comporta como la version sin termino mecanico, bit a bit — que es exactamente lo que hace MATLAB cuando el ajuste mecanico esta desactivado.

TERMINOS OPCIONALES (correccion O1) — NINGUNO ENTRA POR OMISION

`Estudio_Discriminadores` midio que el objetivo iguala lo que no distingue y deja fuera lo que si:

Tb.Sp 0.6 sigma — es una FORMULA, Tb.Th (1/BV.TV - 1), de dos terminos que el ajuste ya iguala H desv (curvatura) 64 sigma Po.Dm medio 39 sigma Tb.Th CV 23 sigma frac. placas EF 36 sigma

y la literatura reciente compara distribuciones (Xiao et al. 2025, distancia de Hellinger entre distribuciones de placas y barras) y el tensor de rigidez entero (Wang et al. 2025; Otto et al. 2025, perdida logaritmica). El argumento `pesos` da acceso a todo eso:

- sustituye el peso de cualquier termino base —{"TbSp": 0} lo quita—;
- activa terminos de `TERMINOS OPCIONALES` con el peso que se les de.

`pesos=None` (por omision) recorre EXACTAMENTE el mismo camino de siempre, en el mismo orden de sumas, y el error es bit a bit el validado contra MATLAB (`validar_error.py`, 1173 pares; el bloque 17 de `tests/` lo repite). Un termino opcional ausente en alguno de los dos dicts se salta y renormaliza (C6), igual que los base.

Como se mide cada termino, para que sumen en la misma escala que las diferencias relativas al cuadrado de los terminos base:

escalar $((b - a) / |a|)^2$ distribucion Wasserstein: $(W1(a, b) / |media\ de\ a|)^2$. Si b es a escalada por un factor s, vale $(s - 1)^2$, lo mismo que un escalar. Hellinger: H^2 , en $[0, 1]$. tensor $d_{LE}(a, b)^2 / 6$, con d_{LE} la distancia log-euclidea entre los tensores en sus ejes materiales. Si $b = s a$, vale $(\ln s)^2$, que para s cerca de 1 es $(s - 1)^2$.

Los terminos mecanicos opcionales NO se multiplican por `peso_mecanico`: su peso es el que se les da en `pesos`. `peso_mecanico` sigue gobernando solo `Ez_rel` y `Ez_Ex`, como en MATLAB.

Funciones

`validar_pesos(pesos)`

Comprueba `pesos` y lo devuelve como dict de floats (o None).

Un nombre mal escrito se rechaza: con un dict permisivo, {"PoDM": 1} no activaria nada y el ajuste parecia haber usado el termino.

`medidas_necesarias(pesos)`

Que hay que medir para poder evaluar estos pesos.

Devuelve {"forma": set, "caro": set, "mecanico": bool}: las compuertas de `metricas_forma` que hacen falta en cada evaluacion, las que solo se pagan entre finalistas, y si algun termino necesita homogeneizar.

`wasserstein_cuantiles(qa, qb)`

Distancia de Wasserstein-1 entre dos distribuciones 1D.

$W1 = \int_0^1 |F_a^{-1}(p) - F_b^{-1}(p)| dp$, por la regla del punto medio sobre los niveles de Hazen. Si las tablas tienen distinto numero de niveles, se lleva la corta a los de la larga.

`hellinger_cuantiles(qa, qb, nbins=32)`

Distancia de Hellinger entre dos distribuciones 1D, en $[0, 1]$.

Se reconstruyen las funciones de distribucion desde los cuantiles, se reparten en `nbins` clases comunes sobre el rango conjunto y $H = \sqrt{1 - \sum \sqrt{p_a p_b}}$. 0 si son iguales, 1 si no se solapan.

`error_morfometrico(m_voi, m_spin, peso_mecanico=0.0, peso_da2=0.0, pesos=None, distancia='wasserstein')`

Error morfometrico ponderado entre VOI y candidato.

Devuelve (err, n_usados). `err` es un promedio ponderado de diferencias relativas al cuadrado; vale PENALIZACION si no hubo ningun termino utilizable.

`peso_mecanico` : equivalente de `localMechFitWeight()`. Escala los pesos de `Ez_rel` y `Ez_Ex`, que salen de la homogeneizacion periodica (`spinpy.fit.metricas_mecanicas`). Si se pone > 0 pero esos campos no estan en los dicts, simplemente no se usan y el error se renormaliza por el peso realmente aplicado (invariante C6): un termino ausente no puede contar como un termino con error cero.

En el ajuste no se activa dentro de la busqueda sino en una etapa de desempate entre finalistas — homogeneizar las ~53 evaluaciones seria entre tres y cuatro ordenes de magnitud mas caro. Ver `ajustar_spinodoide`.

`peso_da2` : peso del termino DA2 (anisotropia secundaria, autovalor mayor entre el intermedio del MIL). NO esta en MATLAB y vale 0 por omision, de modo que el error del spinodoide sigue siendo bit a bit el validado. Existe para el dual-lattice: el DA escalar no distingue una fabrica axisimetrica ($DA2 \sim DA$, lo que da alargar un solo eje) de una triaxial como la del hueso equino (DA 1.45-1.51, $DA2$ 1.12-1.18), y sin este termino un preset triaxial con el mismo DA solo empata con el axial (Estudio_Familias, 4.2 y recomendacion 5).

`pesos` : dict opcional (correccion O1, ver la cabecera del modulo). Sustituye pesos base por nombre y activa terminos de `TERMINOS OPCIONALES`. None deja el camino validado contra MATLAB intacto.

`distancia` : "wasserstein" (por omision) o "hellinger", para los terminos `dist_*`.

`tiene_eje(m)`

Port de `localTieneEje`: ¿el struct trae una direccion principal usable?

`fabric_cos(m_a, m_b)`

Coseno del angulo entre direcciones principales, en valor absoluto.

Se compara como EJE y no como vector: el signo de un autovector es arbitrario, de ahi el valor absoluto.

`peso_alineacion(m_voi, da_min=1.15, da_pleno=1.37, w_max=2.0)`

Port de `localFabricAlignWeight`.

Cuanto vale desempatar por orientacion en ESTE VOI. Devuelve 0 —es decir, no desempatar— cuando la direccion principal no significa nada:

- DA por debajo de 1.15: por debajo del suelo de ruido del MIL (~1.07) la direccion principal es ruido, no estructura.
- MIL acotado: en estructuras laminares el elipsoide degenera y el DA es solo una cota inferior.

Entre 1.15 y 1.37 el peso sube linealmente hasta el maximo.

5 funcion(es) interna(s) de este modulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz publica.

Ajuste: busqueda escalonada y desempate mecanico

`fit.py`

`fit.py` — Ajuste de un spinodoide a un VOI por busqueda escalonada.

Port de `fitSpinodoidToVOI` (AppFinal_V2.m:3407-3720), incluidas las correcciones G1, G2 y K2, que son la parte mas reciente del trabajo sobre la app y la que motivo la version V2.

CORRECCION L1 — LA ORIENTACION SE MIDE, NO SE SUPONE

Es la PRIMERA correccion que no viene de MATLAB: AppFinal_v2.m no la lleva. Quien contraste contra MATLAB tiene que pasar `alinear_fabrica=False`, y asi lo hace `validar_ajuste.py`.

L1. `R_fit` salia de `dir_a_euler(m_voi['dir_principal'])`, que lleva el eje z DEL CANDIDATO sobre la direccion principal del VOI. Eso presupone que el eje z del candidato es su eje rigido, y no lo es: con muestreo por rechazo el eje rigido es el del cono ESTRECHO, y con muestreo equitativo la regla se INVIERTE. Ademas, con dos conos estrechos iguales —[15,15,X], que son 11 de los 14 presets— no hay eje rigido sino un PLANO, y su direccion principal es degenerada.

Consecuencia medida sobre el VOI proximal de H4: el ajuste de menor error dejaba el eje rigido del candidato a 86 grados del hueso y el modulo aparente en la direccion de carga salia 57 veces menor. El DA coincidia (1.456 frente a 1.480) porque el DA es un ESCALAR y no distingue "rigido en Z" de "rigido en Y", y ningun termino del error mira la orientacion.

-> `alinear_por_fabrica`: se genera el ganador sin rotar, se MIDE su tensor de fabrica, se construye la rotacion minima que lleva esa direccion sobre la del VOI, se regenera y se vuelve a medir para comprobar donde quedo.

CORRECCION L2 — LA ALINEACION SE IMPONE, NO SE INTENTA

L1 se RENDIA en tres casos y devolvía entonces la rotacion clasica, que es precisamente la que dejaba el eje perpendicular: si el candidato tenia la fabrica degenerada en un plano (los once presets [15,15,X]), si tras rotar el eje quedaba peor que antes (el generador remuestrea las ondas), o si el candidato era casi isotropo. `avisos.py` lo midio despues en el banco: en 10 de 12 VOIs equinos el eje quedaba a mas de 30 grados.

-> `alinear_marco_fabrica` (por omision, `alineacion="impuesta"`):

- alinea el MARCO MIL entero —cada autovector del candidato con el correspondiente del VOI—, no un solo eje. Un candidato plano tiene normal bien definida aunque no tenga eje principal, y alinear el marco la coloca;
- compara el eje que SIGNIFICA algo: la direccion principal, o la normal del plano si el candidato es plano;
- itera —regenerar, medir, corregir— hasta quedar dentro de `tol_deg` o agotar `iter_max`;
- y se queda con la MEJOR orientacion medida entre la clasica y las iteradas. Nunca devuelve algo peor que la clasica, y dice si llego a la tolerancia (`alineada`). `alineacion="eje"` conserva L1 tal cual.

LO QUE ESTAS CORRECCIONES NO ARREGLAN

El hueco de RIGIDEZ entre hueso y spinodoide es de dos causas, y la orientacion solo toca una. La otra es de la familia: el exponente de Gibson-Ashby medido para los spinodoides es $n \sim 4.1$, frente a $n \sim 2$ del hueso trabecular. A $\rho \sim 0.26$ eso es un factor de 3 a 6 que ninguna rotacion cierra.

LAS TRES ETAPAS

A. densidad x numero de onda (angulos fijos en el valor de partida) B. presets de angulos conicos (con los mejores parametros de A) C. refinado de densidad y numero de ondas (solo en modo completo) D. desempate entre finalistas (solo si hay terminos mecanicos o caros)

Las correcciones que NO se pueden perder al portar:

G2 — el numero de onda se recorre en TODO su rango util [8, 25] en el modo completo, con independencia de donde estuviera el deslizador. La version anterior usaba una ventana [base-5, base, base+5]; con el valor por defecto (15) eso daba [10 15 20] y NUNCA alcanzaba 25, de modo que en trabeculas muy finas el ajuste sobreestimaba Tb.Th en torno al 43 %.

G1 — los angulos conicos se BARREN en la etapa B. Antes quedaban fijos en el valor de los deslizadores y el spinodoide salia casi isotropo (DA ~ 1.1 -1.2), incapaz de alcanzar el DA 1.4-1.5 del hueso proximal equino.

K2 — la lista de presets cubre las PERMUTACIONES, no solo un octante. thetas no es simetrico: el angulo grande marca el eje en torno al cual se concentran los vectores de onda, y por tanto el eje BLANDO. Sin las permutaciones solo se podia orientar la estructura de una manera. Ademas se desempata por orientacion entre los presets cuyo error ya esta dentro del 5 % del mejor (ver `_desempatar_theta`).

FUNCION OBJETIVO E INCERTIDUMBRE (correcciones O1 y U1)

`pesos` y `distancia` pasan tal cual a `error.error_morfometrico` (ver su cabecera). Donde se paga cada termino lo decide su coste: los de forma en cada evaluacion, el Ellipsoid Factor y los mecanicos solo entre finalistas (etapa D). Con `pesos=None` la busqueda es la de siempre, bit a bit.

`replicas` (5 por omision) mide el ganador con K semillas nuevas y reporta el error medio +- sd y el suelo autoconsistente (`spinpy.incertidumbre`). No cambia que candidato gana. `seleccion="robusta"` si lo cambia: elige entre los finalistas por media + `k_robusto` sd sobre esas K semillas.

POR QUE LA ORIENTACION NO ES UN TERMINO DEL ERROR

Se probó y se descarto: sobre los tres VOI de H4 con 5 replicas mejoraba la rigidez pero subia el error de las siete metricas de 0.00726 a 0.00990 (+36 %). Un sumando nuevo compra su objetivo con el de los demas y no hay forma de acotar el intercambio desde dentro de la suma. Aqui la orientacion es criterio SECUNDARIO entre candidatos ya buenos: la morfometria queda acotada por construccion y la formula del error no se toca.

Funciones

```
metricas_mecanicas(BW, spacing, res_mec=16, E_s=E_S_DEF, nu_s=NU_S_DEF)
```

Constantes elasticas de una estructura, por homogeneizacion periodica.

`Ez_rel` y `Ez_Ex` son los dos campos mecanicos historicos que `error_morfometrico` usa con `peso_mecanico > 0`: la rigidez axial normalizada y la anisotropia elastica. Ademas se devuelven, en los EJES MATERIALES y ordenadas $E1 \geq E2 \geq E3$, las constantes de ingenieria (E^*_{rel} , G^*_{rel} , ν^*), el tensor normalizado en esos ejes (`C_principal_rel`, para la distancia log-euclidea), la clase de anisotropia y el residuo del solver.

La estructura se REMUESTREA a `res_mec` antes de homogeneizar porque el coste crece con el cubo del lado: a 16^3 son ~ 6 s y a 32^3 ~ 99 s. En un desempate entre finalistas eso es la diferencia entre medio minuto y media hora.

Devuelve {} si la homogeneización falla, y no ceros: `error_morfometrico` salta los campos ausentes y renormaliza por el peso realmente usado (invariante C6), mientras que un cero fingiría una estructura sin rigidez y arrastraría el ajuste hacia geometrías degeneradas.

Ez_rel DEPENDE FUERTEMENTE DE `res_mec`, y hay que contar con ello. Medido sobre el mismo VOI sintético (48^3, BV/TV 0.345): Ez_rel = 0.108 a 12^3 y 0.064 a 16^3. No es un error de cálculo sino el mismo fenómeno que hace no convergente a E_app en el ensayo de compresión —al remuestrear cambia que trabéculas quedan conectadas— y por eso el número **no es citable como rigidez del espécimen**.

Para lo que sí sirve es para comparar: en el ajuste, VOI y candidatos se homogeneizan todos al MISMO `res_mec`, de modo que el sesgo actúa sobre los dos lados y se cancela en buena parte al restar. Es exactamente el argumento de la corrección C4, y es la única razón por la que un `res_mec` barato es aceptable aquí y no lo sería para reportar un módulo.

```
dir_a_euler(dir_vec)
```

Port de `dirVectorToEulerApprox` (AppFinal_V2.m:5414).

Ángulos de Euler que llevan el eje z sobre `dir_vec`. Se fuerza la componente z positiva porque la dirección principal es un EJE: su signo es arbitrario y quedarse con el hemisferio superior evita rotaciones de 180 grados que no cambian la geometría.

```
alinear_por_fabrica(m_voi, res, sp_cand, dens, wave, nw, thetas, esquema='rechazo',
seed=SEMILLA_POR_DEFECTO, precision='f32', da_min=1.1, da2_max=1.06)
```

Rotación que alinea la fábrica MEDIDA del candidato con la del VOI.

Es la corrección L1, que se conserva como `alineacion="eje"`. Por omisión el ajuste usa `alinear_marco_fabrica` (L2), que no se rinde en los casos en que esta sí (ver la cabecera del módulo).

CORRECCIÓN L1 — POR QUE NO VALE SUPONER QUE EL EJE RÍGIDO ES Z

Hasta aquí, `R_fit` salía de `dir_a_euler(m_voi['dir_principal'])`, que construye la rotación que lleva **el eje z del spinodoide** sobre la dirección principal del VOI. Eso presupone que el eje z del candidato es su eje rígido, y NO LO ES en general:

- Con muestreo por RECHAZO, el eje rígido es el del cono ESTRECHO: el cono ancho se lleva casi todas las ondas por ángulo sólido, el campo oscila rápido en esa dirección y esa dirección queda blanda. Medido: `thetas` [45,45,15] -> rígido en Z; [45,15,45] -> rígido en Y; [15,45,45] -> rígido en X. Tres de tres.
- Con muestreo EQUITATIVO la regla SE INVIERTE: el cono estrecho recibe su tercio de ondas concentrado junto a su eje, y es ese eje el que se ablanda. Medido: [45,45,15] -> rígido en Y; [90,90,15] -> rígido en X.
- Y con dos conos estrechos iguales ([15,15,X]) no hay eje rígido sino un PLANO rígido: la dirección principal es degenerada y alinearla no significa nada.

Consecuencia medida sobre el VOI proximal de H4: el ajuste de menor error dejaba el eje rígido del candidato a 86 grados del que tiene el hueso, y el módulo aparente en la dirección de carga salía 57 veces menor que el del VOI. El DA coincidía (1.456 frente a 1.480) porque el DA es un ESCALAR y no distingue "rígido en Z" de "rígido en Y".

Como ningún término de `error_morfometrico` depende de la orientación, la corrección no cambia que candidato gana: solo lo coloca bien. Por eso se aplica DESPUÉS de elegir los parámetros y cuesta dos generaciones extra.

COMO SE HACE

- 1 Se genera el candidato SIN rotar y se mide su tensor de fábrica.
- 2 Se construye la rotación mínima que lleva esa dirección medida sobre la del VOI.
- 3 Se regenera con ella y se VUELVE A MEDIR, porque el generador remuestrea las ondas y el resultado no es una rotación exacta del anterior. El ángulo que queda se devuelve para que conste.

CUANDO NO SE APLICA Si el VOI o el candidato son casi isotropos ($DA < da_min$) la direccion principal es ruido, y si el candidato tiene la fabrica degenerada en un plano ($DA2 < da2_max$) no hay un eje que alinearse. En los dos casos se devuelve la rotacion clasica y se dice por que.

Devuelve (R, info).

```
alinearse_marco_fabrica(m_voi, generar, medir, R_clasica, m_clasica=None, da_min=1.1,
da2_plano=1.06, tol_deg=10.0, iter_max=3)
```

Correccion L2: impone la orientacion del marco MIL del VOI al candidato.

generar(R) -> mascara del ganador con la rotacion R medir(mascara) -> dict de morfometria (con eigenvectors, DA, DA2) m_clasica : la medida del ganador con R_clasica, si ya se tiene (el ajuste la trae de la busqueda); si no, se genera.

Sirve a las dos familias: en el dual-lattice la rotacion es exacta y suele bastar una iteracion; en el spinodoide el generador remuestrea las ondas al rotar y la iteracion corrige lo que quede.

El eje que se compara es la direccion principal (autovector del menor autovalor del MIL, el de MIL maximo) salvo que el candidato sea PLANO ($DA2 < da2_plano$: los dos primeros autovalores iguales), en cuyo caso se compara la NORMAL del plano, que es lo que ese candidato si tiene bien definido.

No se aplica —y se dice— si el VOI o el candidato son casi isotropos: sin eje no hay orientacion que imponer ni que perder.

Devuelve (R, info). info conserva las claves de L1 (aplicada, motivo, angulo_antes_deg, angulo_despues_deg, metodo) y anade alineada, iteraciones, eje_comparado e historial_deg.

```
longitud_caracteristica(voi_shape, spacing)
```

Port de localCharacteristicLength: lado fisico medio del VOI, en mm.

Correccion F1: el spinodoide se genera en un cubo unidad adimensional. Si no se escala al tamano fisico del VOI antes de medirlo, Tb.Th, Tb.Sp y BS/BV salen en unidades distintas y restarlas no significa nada.

```
resolucion_comparacion(voi_shape)
```

Port de localVoxelCountForComparison: $24 \leq n \leq 96$, tomado del VOI.

```
fraccion_poro_conexa(BW)
```

Fraccion del poro que pertenece a su mayor componente conexa (caras).

Mide si la estructura sigue siendo BICONTINUA. En hueso trabecular la medula forma una red conectada; cuando este valor cae, la estructura ha pasado a ser un solido con poros aislados y deja de ser trabecula, por bien que coincida el BV/TV.

```
validar_opciones(pesos, distancia, replicas, seleccion, alineacion)
```

Comprueba las opciones de objetivo, incertidumbre y alineacion.

Compartida con fit_dual: una opcion mal escrita tiene que fallar al empezar, no despues de media hora de busqueda.

```
claves_incertidumbre(pesos, usa_mec)
```

Sin documentar.

```
ajustar_spinodoide(VOI, spacing, modo='completo', m_voi=None, esquema='rechazo',
precision='f32', seed=SEMILLA_POR_DEFECTO, num_waves=700, resolucion=None, progreso=None,
rho_max=RHO_MAX, peso_mecanico=0.0, res_mec=16, n_finalistas=5, E_s=E_S_DEF, nu_s=NU_S_DEF,
alinearse_fabrica=True, pesos=None, distancia='wasserstein', replicas=REPLICAS_DEF,
seleccion='minimo', k_robusto=1.0, alineacion='impuesta')
```

Busca los parametros de spinodoide que mejor reproducen el VOI.

modo : 'completo' (etapas A+B+C, 14 presets) | 'rapido' (A+B, 4 presets)

alinear_fabrica : al terminar la busqueda, la orientacion del ganador se corrige usando su fabrica MEDIDA en vez de suponer que su eje rigido es z. No cambia que candidato gana —ningun termino del error depende de la orientacion— solo lo coloca bien. Ponerlo a False reproduce el comportamiento anterior, que es lo que hay que usar para contrastar contra AppFinal_V2.m, porque MATLAB no lleva esta correccion.

alineacion : 'impuesta' (L2, por omision: marco MIL entero, iterado, nunca peor que la clasica) | 'eje' (L1, la de antes). Ver la cabecera.

peso_mecanico : si es > 0 se anade una ETAPA D que reordena los `n_finalistas` mejores candidatos incluyendo `Ez_rel` y `Ez_Ex`, obtenidos por homogeneizacion a `res_mec`.

POR QUE UNA ETAPA APARTE Y NO DENTRO DE LA BUSQUEDA. Homogeneizar es entre tres y cuatro ordenes de magnitud mas caro que medir morfometria: las ~53 evaluaciones del modo completo pasarian de segundos a horas. La etapa D paga la homogeneizacion solo donde puede cambiar la decision —entre candidatos que ya son buenos morfometricamente— y son `n_finalistas`+1 homogeneizaciones en total, contando la del VOI.

LO QUE ESTO NO ES. No es una busqueda con el termino mecanico dentro: si el optimo mecanico esta en una region que la busqueda morfometrica descarto pronto, la etapa D no lo encuentra. Es un desempate informado, y `traza_mecanica` deja ver cuanto reordeno para que se pueda juzgar si el desempate estaba decidiendo algo o solo confirmando.

pesos, distancia : terminos opcionales del objetivo (correccion O1, ver `error.error_morfometrico`). Los de forma se miden en cada evaluacion; el Ellipsoid Factor y los mecanicos (`C_logE`, `E*_rel`, `G*_rel`) activan la etapa D y solo se pagan entre finalistas. Ojo con el EF: son varios minutos por estructura, tambien sobre el VOI.

replicas : K semillas nuevas con las que se mide el ganador para el informe `incertidumbre` (0 lo desactiva; 1 a 4 es un error).

seleccion : 'minimo' (el de siempre) | 'robusta': entre los finalistas, el de menor media + `k_robusto` sd del error sobre las K semillas.

precision : 'f32' por defecto en la busqueda. Son ~50 evaluaciones y la diferencia frente a f64 no cambia ningun voxel en los casos medidos (ver `spinpy.grf._evaluar_campo`). El candidato ganador se REMIDE al final en f64, de modo que las metricas que se reportan son exactas.

Devuelve un dict con los parametros ganadores, sus metricas y la traza.

`_4` funcion(es) interna(s) de este modulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz publica._

Homogeneizacion elastica periodica

`elastic.py`

`elastic.py` — Homogeneizacion numerica periodica sobre la rejilla de voxeles.

Port de: `localHex8Ke` (AppFinal_V2.m:6121-6167) `localHomogenizeVoxel` (AppFinal_V2.m:6192-6430) `localEngineeringConstants`(AppFinal_V2.m:6452-...)

Resuelve el problema de celda unidad de verdad —elementos hexaedricos sobre la rejilla de voxeles, condiciones de contorno periodicas, seis casos de deformacion macroscopica unitaria— y obtiene

$$C_{ij} = (1/|V|) * \sum_e (u0_i - chi_i)' ke (u0_j - chi_j)$$

siendo `chi` el campo corrector. Referencia: Andreassen & Andreassen, Comput. Mater. Sci. 83 (2014) 488-495.

SUSTITUYE al tensor heuristico que la app tenia antes ($E_{eff} = E_s * \rho^2$ con un reparto lineal segun los angulos y un factor 0.9 sin justificacion), en el que la rigidez no derivaba de la geometria en ningun momento: dos estructuras con la misma densidad daban el mismo tensor aunque su topologia fuera distinta.

CONVENCIONES QUE NO SE PUEDEN CAMBIAR

- Voigt [xx yy zz yz xz xy] con deformaciones angulares de INGENIERIA, en todo el archivo y en todo el proyecto.
- El vacio se modela con rigidez $1e-6 * E_s$ en lugar de eliminarse: evita que islas de material desconectadas hagan singular la matriz, a costa de un sesgo despreciable frente al solido.
- La rejilla de NODOS tiene el mismo tamaño que la de elementos: el nodo $nx+1$ es, por periodicidad, el nodo 1.
- Los arrays se recorren en orden Fortran (columna primero) para que el índice de elemento coincida con el de MATLAB.

Funciones

`hex8_ke(dx, dy, dz, E, nu)`

Rigidez de un hexaedro trilineal de 8 nodos, Gauss $2 \times 2 \times 2$.

Se calcula numericamente en lugar de usar una matriz precomputada: mas largo, pero verificable y sin riesgo de erratas de transcripcion.

`homogeneizar(BW, E_s=1.0, nu_s=0.3, vox_size=1.0, tol=1e-08, escala_vacio=VOID_SCALE, verbose=False)`

Tensor de rigidez homogeneizado C_h (6×6) en orden de Voigt.

Devuelve (Ch, info). Si algo impide resolver, Ch es NaN e `info['msg']` explica por que — nunca se devuelve un tensor silenciosamente equivocado.

`escala_vacio` : rigidez de la fase `False` relativa a la solida. El valor por defecto $1e-6$ es el de la app y representa vacio. Se expone como parametro porque permite VERIFICAR la funcion contra el promedio de Backus con dos fases reales: con vacio a $1e-6$ casi todas las entradas del tensor exacto valen ~ 0 y la comparacion no distingue una implementacion correcta de una equivocada.

`remuestrear_bw(BW, spacing, n_max)`

Reduce la mascara a $n_{\max}$ voxels por eje. Port de `localResampleBW`.

Submuestreo por vecino mas proximo, no por promediado: la mascara debe seguir siendo binaria. El `spacing` se corrige por el factor real alcanzado, no por el pedido, porque `round` sobre `linspace` no da exactamente $n_{\max}$.

El coste de la homogeneizacion crece con el CUBO del lado, asi que este recorte no es cosmetico: es lo que hace viable calcular un tensor sobre un VOI de 97^3 .

`constantes_ingenieria(C)`

Constantes de ingenieria a partir de C (6×6), via $S = \text{inv}(C)$.

Orden de Voigt [xx yy zz yz xz xy] con deformaciones angulares de ingenieria, de modo que $S[3,3] = 1/G_{yz}$ directamente.

`voigt_a_tensor4(C)`

C (6×6 , Voigt con deformaciones angulares de ingenieria) $\rightarrow C_{ijkl}$.

Con esa convencion $C_{IJ} = C_{ijkl}$ sin factores, que es lo que hace la conversion trivial y la razon por la que el proyecto la fija.

`tensor4_a_voigt(T)`

`_Sin documentar._`

`en_ejes(C, V)`

Componentes de C en la base ortonormal cuyas COLUMNAS son V .

$C'_{ijkl} = V_{pi} V_{qj} V_{rk} V_{sl} C_{pqrs}$. Un V con determinante -1 da el mismo tensor que su rotación: un tensor de orden par no cambia por inversión.

`a_mandel(C)`

Voigt de ingeniería $\rightarrow$ Mandel: $C_M = W C W$, $W = \text{diag}(1,1,1,r_2,r_2,r_2)$.

En Mandel el tensor es una matriz 6×6 de verdad —sus autovalores son los de la aplicación lineal— y una rotación es una matriz ortogonal. En Voigt ninguna de las dos cosas es cierta, y por eso el logaritmo se toma aquí.

`distancia_log_euclidea(C1, C2)`

// $\log C1 - \log C2$ // $_F$, con los tensores en notación de Mandel.

PROPIEDADES QUE LA HACEN LA ADECUADA (y que el bloque 17 comprueba):

- es simétrica y vale 0 solo si $C1 = C2$;
- $d(aC, C) = \sqrt{6} |\ln a|$: una estructura el doble de rígida en todo está a la misma distancia que una la mitad, que es lo que pide un error RELATIVO;
- $d(C1, C2) = d(S1, S2)$, porque $\log(C^{-1}) = -\log C$: igualar la rigidez y la flexibilidad a la vez, que es el motivo por el que Otto et al. (2025) eligen una pérdida logarítmica;
- no cambia si se gira a los DOS con la misma rotación.

Lo que NO es: invariante si se gira solo uno. Para comparar sin que cuente la orientación hay que llevar los dos a sus ejes materiales antes (`constantes_principales`), que es lo que hace el ajuste.

`ejes_materiales(C, tol=1e-06)`

Ejes de simetría material de C (columnas de una matriz 3×3).

Se toman de los autovectores del tensor DILATACIONAL $d_{ij} = C_{ijkk}$ o del de VOIGT $v_{ij} = C_{ikjk}$ (Cowin 1987): para un material ortotrope los dos son diagonales en sus ejes de simetría. Se usa el que tenga más autovalores distintos, y el dilatacional en caso de empate.

Con DOS autovalores iguales (material transversalmente isotropo) el autovector distinto es el eje de simetría y los otros dos son una base cualquiera del plano; para ese material cualquier base del plano es exacta, así que no hay nada que decidir. La primera versión exigía tres autovalores distintos y devolvía los ejes del laboratorio para un laminado girado: sus constantes no se recuperaban (bloque 17).

Si los dos tensores son isotropos —material cúbico o isotropo— no fijan ejes y se devuelven los del laboratorio. En las estructuras de este proyecto, que viven en una rejilla de voxels, los ejes cúbicos coinciden con los de la rejilla, así que es el supuesto razonable; se declara con `fuelle`.

Devuelve (V, fuente) con fuente "dilatacional" | "voigt" | "laboratorio".

`constantes_principales(C)`

Constantes de ingeniería en los ejes MATERIALES, ordenados $E1 \geq E2 \geq E3$.

Con eso dos tensores se comparan por lo que son y no por como están colocados: el eje más rígido de uno contra el más rígido del otro. La orientación se trata aparte, en la alineación del ajuste.

`desviacion_ortotropa` es la fracción de la norma (Mandel) que queda en las entradas que un material ortotrope tendría a cero una vez en sus ejes. Si no es pequeña, las nueve constantes no describen el tensor.

`clase_anisotropia(C, tol=0.05)`

Clase de simetría elástica de C , con una tolerancia relativa.

CRITERIO PROPIO, DECLARADO. Otto et al. (2025) proponen un metodo para asignar la clase de anisotropia de un tensor; no se ha podido leer su texto completo, asi que esto NO lo reproduce. Es un criterio por tolerancia sobre las constantes en los ejes materiales:

sin_simetria_ortotropa desviacion_ortotropa > tol isotropa $E_1=E_2=E_3$, G iguales y $G = E / (2 (1 + \nu))$ cubica $E_1=E_2=E_3$ y G iguales, sin la relacion anterior transversal dos E iguales, con sus G y la relacion en el plano ortotropa el resto

"Iguales" es $|a - b| \leq \text{tol} * \max(|a|, |b|)$. Con la dispersion entre realizaciones de un spinodoide (varios por ciento) una tolerancia menor que esa convierte ruido en clase.

`backus_laminado(fracciones, E, nu)`

Tensor EXACTO de un laminado de capas isotropas normales a z.

Promedio de Backus (1962). Es la solucion analitica contra la que se verifica `homogeneizar`: para un apilado de capas planas la solucion de elementos finitos debe reproducirla, porque el campo corrector es constante dentro de cada capa y el hexaedro trilineal lo representa exactamente.

fracciones : fracciones de volumen de cada capa (suman 1) E, nu : modulos y coeficientes de Poisson de cada capa

`_5` funcion(es) interna(s) de este modulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz publica._

Ensayo de compresion, von Mises y Pistoia

`resistencia.py`

`resistencia.py` — Ensayo de compresion axial y carga de fallo aparente.

Cubre lo que a la rigidez le falta: para biomecanica, saber que un hueso es rigido es media historia; la pregunta que suele importar es a que carga rompe.

CRITERIO DE PISTOIA

Pistoia et al. (Bone 30:842, 2002). Sobre una UNICA resolucion LINEAL se busca el factor por el que hay que escalar la carga para que un porcentaje del tejido supere una deformacion critica:

el hueso falla cuando el 2% del volumen oseos supera el 0.7% de deformacion efectiva

Como el problema es lineal, la deformacion escala con la carga y el factor sale de una division: no hace falta analisis no lineal ni iterar. El criterio se valido originalmente contra ensayos de radio distal humano y predice la carga de rotura mucho mejor que la densidad sola.

La deformacion efectiva es la definicion energetica que usa esa literatura:

$\text{eps_eff} = \sqrt{2 * U / E_s}$ con U la densidad de energia de deformacion

Para un estado uniaxial se reduce a $|\text{eps_zz}|$, lo que permite comprobarla contra una solucion exacta.

LOS DOS PARAMETROS SON CONVENCIONES, NO CONSTANTES FISICAS

El 2% y el 0.7% se calibraron en radio distal humano a una resolucion concreta. No son universales: dependen de la especie, del sitio y —esto importa— de la RESOLUCION de la malla, porque el percentil de una cola depende de cuantos elementos haya. Se exponen como argumentos y se reportan siempre junto al resultado, en lugar de esconderlos como constantes.

CONDICIONES DE CONTORNO

'empotrado' base totalmente fija: reproduce un ensayo con fricción entre probeta y platos, introduce coacción lateral y rigidiza. 'deslizante' $uz = 0$ en la base y solo lo mínimo para eliminar los movimientos de sólido rígido: compresión sin fricción, estado uniaxial.

La elección cambia E_{app} y la carga de fallo, así que hay que declararla.

TRES CAMPOS DE SALIDA, NO UNO

Del mismo campo de tensiones salen `campo_eps_eff` (deformación efectiva de Pistoia, energética) y `campo_vm` (von Mises, solo la parte desviadora). No son el mismo mapa reescalado: coinciden salvo un factor en estado uniaxial y se separan en los nudos trabeculares, donde el estado es triaxial. Uno predice la carga aparente de fallo y el otro localiza donde plastificaría el tejido.

El tercero es `campo_desp`, la DEFORMACIÓN TOTAL: el módulo del vector desplazamiento, $|u| = \sqrt{u_x^2 + u_y^2 + u_z^2}$, en las unidades de `spacing`. Es la variable "Total Deformation" que reporta ANSYS y la que usan los estudios comparativos microCT-FEA junto a la von Mises. No es una tensión ni una deformación unitaria: es un desplazamiento, y su máximo está siempre en la cara cargada porque acumula todo lo que se ha movido la probeta. Mide RIGIDEZ GLOBAL, mientras la von Mises localiza la concentración de tensión.

TRES EJES

`ensayo_compresion_eje` corre el mismo ensayo en X, Y o Z permutando la estructura, y `ensayo_triaxial` los tres de una vez. El cociente E_{max}/E_{min} que sale de ahí es una anisotropía MECÁNICA, contrastable con $E_x/E_y/E_z$ de la homogeneización periódica y distinta del DA del MIL, que es geométrico.

Funciones

`normalizar_apoyo(apoyo)`

Devuelve la clave canónica de `apoyo` o lanza `ValueError`.

Admite mayúsculas y espacios alrededor, nada más: ni traducciones ("fixed", "sliding") ni prefijos. Un apoyo mal nombrado cambia E_{app} y la carga de fallo sin dar ningún síntoma, así que vale más un error.

`percentil_ponderado(v, w, q, metodo='hazen')`

Percentil ponderado por w (volumen de cada elemento).

Con voxels todos los elementos pesan lo mismo y el percentil por conteo YA está ponderado por volumen. Con tetraedros no: «el 2 % del volumen óseo» (Pistoia) o el p99 de la capa superficial tienen que contar volumen, no elementos.

DEFINICIÓN. Se ordenan los valores; cada elemento ocupa un tramo de peso w_i y se sitúa en su punto MEDIO, $c_i = S_i - w_i / 2$ (S_i , peso acumulado). Entre esos puntos se interpola linealmente y fuera se toma el extremo: 'hazen' posición c_i / W : con pesos iguales es $(i - 0.5)/n$, la de `prctile` de MATLAB (Q5 de Hyndman y Fan). 'linear' posición $(c_i - c_{-1})/(c_n - c_{-1})$: con pesos iguales es $(i - 1)/(n - 1)$, el defecto de numpy (Q7), que es el que usan el umbral de Pistoia y el `vm_p99` de todo el tejido.

Con pesos iguales (o w None) se llama a `np.percentile` con el mismo método, de modo que el resultado es BIT A BIT el de siempre: el bloque 27 lo comprueba, y comprueba también que la fórmula ponderada con pesos iguales coincide con numpy a redondeo.

`capa_superficie(BW)`

Elementos sólidos con al menos un vecino POR CARA en el vacío.

Es la capa donde vive la escalera de la voxelización y donde se concentra la tensión. Importa porque el máximo de von Mises NO CONVERGE con la resolución —ni siquiera contra solución cerrada: en la cavidad esférica periódica el pico binario se pasa un +5.6 % del valor de Goodier a la resolución más fina y va dando saltos de ~10 puntos por el

camino (`Estudio_Convergencia`)—, mientras que un percentil 99 TOMADO SOBRE ESTA CAPA se queda a -0.3 % del exacto. Un p99 sobre TODO el tejido tampoco sirve en estructuras densas: lo domina el material a granel, donde la tension es la nominal, y no dice nada del pico.

OJO — ESTA DEFINICION NO ES PERIODICA, y la de `Estudio_Convergencia/code/conv_nucleo.py` SI LO ES. Alli la celda se homogeneiza con condiciones periodicas, asi que un vecino que sale por una cara entra por la opuesta y `np.roll` es lo correcto. Aqui el ensayo tiene base y techo: envolver compararia la cara cargada con la apoyada y marcara como superficie libre justo los elementos donde la condicion de contorno concentra tension por si misma. Fuera del dominio se supone material, de modo que las seis caras del VOI no cuentan como interfaz —la misma regla que excluye las isocaps del area superficial en `morphometry`—. Las dos definiciones coinciden en el interior y solo se separan en el borde; no son intercambiables, y por eso no se comparte el codigo.

`estadisticos_vm(res)`

Estadisticos de von Mises en la capa superficial de un ensayo resuelto.

Toma el diccionario que devuelve `ensayo_compresion`. Usa `vm_solido` y la mascara `superficie_solido` que ese mismo ensayo deja preparada, las dos en el mismo orden y en doble precision. Para resultados ANTIGUOS, que no llevan esa mascara, la deduce de `campo_vm`; sale lo mismo salvo que el campo se guarda en float32, asi que los valores pueden diferir en la septima cifra. Se avisa aqui porque si no, un `vm_max_superficie` que no coincide exactamente con `vm_max` parece un error y no lo es.

Devuelve `vm_p99_superficie` —el estadistico que el estudio de convergencia valida— y, para poder juzgarlo, `vm_n_superficie`: un percentil de la cola de una muestra pequena es ruido. Un dominio completamente macizo no tiene superficie libre interior y devuelve NaN con `n = 0`; es correcto, no hay concentracion que medir.

Los percentiles de aqui usan la convencion de `prctile` (ver `_pct`).

Si `res` trae `vol_solido` —el volumen de cada elemento, alineado con `vm_solido`; lo ponen los ensayos en FEBio con tetraedros— el percentil, la media y la fraccion se PONDERAN por volumen (`percentil_ponderado`). Con volúmenes iguales, o sin la clave, el resultado es bit a bit el de siempre.

`cuantiles_vm_superficie(res, probs=PROB_CUANTILES_VM, escala=1.0)`

Distribucion de von Mises en la capa superficial, como cuantiles.

Misma muestra que `estadisticos_vm` —el tejido de `superficie_solido`— y misma convencion de percentil (hazen, la de `prctile`), de modo que el cuantil 99 de la lista coincide exactamente con `vm_p99_superficie`. Es lo que permite dibujar la cola de la que sale el valor citable a partir del JSON exportado, sin guardar el campo de cientos de MB.

`escala` multiplica los valores (1e-6 para pasar de Pa a MPa). Devuelve {"p": [...], "valor": [...], "n": int} o {} si no hay capa superficial.

`ensayo_compresion(BW, spacing, E_s=E_S_DEF, nu_s=NU_S_DEF, sigma0=SIGMA0_DEF, apoyo='deslizante', escala_vacio=None, tol=1e-08, solo_portante=True, carga_N=None, unidad='mm', progreso=None, modulo_rel=None)`

Compresion axial en z. Devuelve campos y magnitudes aparentes.

`modulo_rel` (opcional, misma forma que `BW`): rigidez relativa de cada voxel de hueso, en (0, 1]. Con None —el valor por omision— todo el hueso tiene `E_s` y el resultado es identico al de siempre. Existe para el fallo progresivo (`spinpy.simulacion.fallo_progresivo`), que ABLANDA el tejido roto en lugar de borrarlo: borrar puntales de uno o dos elementos dejaba fragmentos casi como mecanismos y la rigidez subia y bajaba entre pasos (medido sobre el VOI proximal de H4 a 32^3). La tension se escala con el mismo factor, asi que la energia y `eps_eff` de un elemento ablandado bajan en proporcion.

La tension aparente `sigma0` se define sobre la seccion BRUTA del VOI (lado x lado), no sobre el area osea. Es la convencion de la app y la que hace comparables los resultados con la morfometria: aplicarla sobre el area real daria

una tension mucho mayor y dependiente del BV/TV.

`carga_N` es la alternativa: fuerza TOTAL en newtons, de la que se deduce $\sigma_0 = \text{carga_N} / A_{\text{bruta}}$. Existe porque los estudios comparativos microCT-FEA aplican una fuerza fija (100 N es lo habitual) y no una tension. OJO AL COMPARAR: a fuerza fija, dos VOIs de distinta seccion reciben tensiones aparentes distintas; a tension fija reciben fuerzas distintas. Si los dos volumenenes tienen el mismo lado -que es el caso al comparar un VOI con su spinodoide ajustado, escalado al tamaño físico del VOI- las dos convenciones coinciden. Si no, hay que declarar cual se uso.

UNIDADES — `unidad` SOLO importa si se da `carga_N`

El resto de la funcion es indiferente a las unidades: la deformacion es adimensional, así que las tensiones salen en las mismas unidades que `E_s` y los desplazamientos en las mismas que `spacing`. Una fuerza, en cambio, mezcla las dos: $\sigma = F / A$ obliga a que `F`, `E_s` y `spacing` pertenezcan al mismo sistema.

En esta aplicacion `spacing` esta en MILIMETROS y `E_s` en PASCALES, que no es un sistema coherente. Para que `sigma` salga en Pa a partir de un area en mm^2 hay que multiplicar la fuerza en newtons por $1e6$:

$\sigma[\text{Pa}] = F[\text{N}] / A[\text{m}^2] = 1e6 * F[\text{N}] / A[\text{mm}^2]$

`unidad='mm'` (por defecto) aplica ese factor; `unidad='m'` no lo aplica y corresponde a un sistema SI puro. Sin esto, pedir 100 N con el `spacing` en mm aplica en realidad $1e-4$ N y todo sale un millon de veces mas pequeno: von Mises de 0.000 MPa y desplazamientos de 0.00000 mm.

EL VACIO NO SE MALLA — y por que importa

homogeneizar rellena los poros con un material de rigidez $1e-6 * E_s$ para que la celda periodica nunca sea singular. Aqui eso NO funciona: medido sobre el VOI proximal de H4 a $32^{\wedge}3$ (BV/TV 0.28), el solver se quedaba en un residuo de $3.3e-1$ —sin converger en absoluto— porque el contraste de $1e6$ entre hueso y relleno arruina el condicionamiento en una estructura poco densa.

La salida es la de la practica habitual en micro-elementos finitos (Van Rietbergen): mallar SOLO el hueso y reenumerar los nodos. El sistema queda bien condicionado mientras el hueso este conectado, que es justo lo que garantiza `solo_portante`. Ademas es mas rapido: a BV/TV 0.28 hay un 72% menos de elementos.

`escala_vacio` se conserva por compatibilidad; si se le da un valor, se vuelve al modelo antiguo con relleno blando.

`criterio_pistoia(res, frac=FRAC_CRITICA, eps_crit=EPS_CRITICA)`

Carga de fallo aparente a partir de un ensayo lineal ya resuelto.

Devuelve el factor por el que hay que escalar la carga para que la fraccion `frac` del volumen oseó alcance `eps_crit` de deformacion efectiva, y la tension y carga de fallo que resultan.

Como el problema es LINEAL, `eps` escala con la carga y el factor es una division. Toda la maquinaria no lineal que uno esperaria aqui sobra: esa es justamente la gracia del criterio.

`ensayo_compresion_eje(BW, spacing, eje=2, **kw)`

`ensayo_compresion` a lo largo de X, Y o Z (`eje = 0, 1 o 2`).

El ensayo esta escrito para comprimir en z —el filtro portante, la cara cargada, el apoyo y `E_app` miran todos a la tercera dimension—. En vez de duplicar esa logica con indices parametrizados, que es donde se cuelan los errores que no dan error, se PERMUTA la estructura para que el eje pedido ocupe la posicion z, se resuelve el mismo codigo de siempre y se deshace la permutacion en los campos 3D.

QUE MARCO USA CADA SALIDA — importa al leerla:

- `E_app`, `eps_app`, `sigma_fallo`, `frac_portante` y todo lo escalar son invariantes o ya estan referidos al eje ensayado: se leen tal cual.
- `campo_eps_eff`, `campo_vm` y `campo_desp` se devuelven en el marco ORIGINAL, listos para pintar sobre la geometria sin tocar nada.
- `eps`, `sig` y `u` se quedan en el marco DEL ENSAYO (donde la carga va en z). Deshacer esa rotacion componente a componente seria facil de equivocar y ninguna parte del visor los usa, asi que se marcan con `marco_permutado` en vez de convertirlos a medias.

```
ensayo_triaxial(BW, spacing, ejes=(0, 1, 2), progreso=None, **kw)
```

Ensayo de compresion en los tres ejes. Devuelve un dict por eje.

PARA QUE SIRVE: `E_app` de un solo eje no dice si la estructura es anisotropia; con los tres sale un cociente `E_max/E_min` que es una medida MECANICA de anisotropia, independiente del DA del tensor MIL —que es puramente geometrico— y comparable con `Ex`, `Ey`, `Ez` de la homogeneizacion periodica de `spinpy.elastic`.

Que las dos rutas coincidan no es trivial: la homogeneizacion impone contorno PERIODICO sobre la celda y este ensayo impone platos reales sobre una probeta finita. Coinciden solo si la estructura es grande frente a la trabecula. Discrepar es informacion —efecto de tamaño o de borde—, no un fallo; por eso conviene mirar las dos y no sustituir una por la otra.

La clave `resumen` trae `E_app` por eje y la anisotropia, ya calculados.

```
estudio_convergencia(BW, spacing, resoluciones=(20, 24, 28, 32, 40), eje=2,  
remuestrear=None, progreso=None, **kw)
```

`E_app` a varias resoluciones, para decidir si el valor es citable.

POR QUE HACE FALTA. En micro-elementos finitos sobre estructuras poco densas `E_app` NO converge sin más al refinar: medido sobre el VOI proximal de H4 (BV/TV 0.28) salieron 700, 797 y 365 MPa a $32^{\wedge}3$, $48^{\wedge}3$ y $64^{\wedge}3$. Un valor tomado de una sola resolución no significa nada, y esa advertencia ya estaba escrita en la interfaz sin que hubiera forma de resolverla.

QUE SIGNIFICA UNA SERIE NO MONOTONA. No es ruido numérico —el problema es determinista y el residuo del solver se comprueba en cada punto—: es que al refinar cambia la GEOMETRIA. Cada remuestreo redibuja qué trabéculas quedan conectadas, y una trabécula de uno o dos vóxeles puede aparecer, desaparecer o partirse. El modelo no está convergiendo hacia una estructura fija; está resolviendo estructuras distintas. Por eso se devuelve también `rho` y `frac_portante` de cada punto: si varían, la comparación de `E_app` entre resoluciones no es una comparación de discretización.

Se devuelve además el orden de convergencia observado entre los tres últimos puntos y la extrapolación de Richardson, PERO solo cuando la serie es monótona y la densidad estable; en caso contrario se marcan como no aplicables en vez de dar un número que invitaría a citarlo.

8 funcion(es) interna(s) de este modulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz publica.

Ensayos FEM con motores internos (hex8 y TET10)

`fem.py`

`fem.py`: Ensayos de elementos finitos de spinpy con motores INTERNOS.

Sustituye a la integracion con FEBio (`febio4.exe`, un proceso externo). El mallado, los protocolos, el postproceso y las tablas son los mismos que se validaron contra FEBio (`comparativa_febio/`, `comparativa_febio_tet/`); lo que cambia es quien resuelve: uno o varios de los motores de `spinpy.motores` (la app, NGSolve, FEniCSx, scikit-fem, SfePy), elegidos por el usuario y comparados en `comparativa_motores/INFORME.md`.

Dos mallas posibles sobre la MISMA mascara:

hex8 la malla de voxels de la app, un elemento por voxel. Con el motor 'app' es exactamente `resistencia.ensayo_compresion`; con los demas motores es el mismo problema discreto (la comparativa lo verifica a $\sim 1e-10$), y ademas admite plato rigido y no linealidad geometrica. tet10 una malla tetraedrica cuadratica SUAVE (`solido.malla_tet10`) que quita los escalones. La app no la resuelve; si los demas motores.

Con varios motores sobre la misma malla y el mismo protocolo, la diferencia entre motores mide la IMPLEMENTACION (debe ser ~ 0); la diferencia entre hex8 y tet10 con un mismo motor mide solo el efecto de la MALLA.

DIFERENCIAS CON LA RUTA DE FEBIO (declaradas)

- Lineal: se resuelve el problema LINEAL directamente, una sola vez. FEBio es no lineal geometricamente y habia que extrapolar a carga nula con dos cargas pequenas ($u = 2 u(s) - u(2 s)$).
- Fuerza de reaccion: sale del vector de fuerzas internas en el techo. FEBio 4.5 escribia reaccion cero en los `zero displacement` y se reconstruia por la integral de volumen de `sigma_zz`, que aqui queda como COMPROBACION de equilibrio (`dF_rel`).
- No lineal con fuerza: la traccion es MUERTA (fija en la configuracion de referencia). La presion de FEBio seguia a la cara deformada (seguidora); la diferencia es del orden de la rotacion de las caras del techo, pequena a las cargas de los protocolos, pero no nula.

LA GUI SOLO LLAMA A ESTE MODULO: todo corre igual desde la CLI y los tests.

Unidades: E y sigma viajan en Pa, como en el resto de spinpy; los motores trabajan en mm - N - MPa y la conversion se hace aqui.

Funciones

`von_mises(s)`

von Mises = $\sqrt{3 J_2}$ de tensiones Voigt $[xx \ yy \ zz \ yz \ xz \ xy]$.

`eps_eff(sig, E, nu)`

Deformacion efectiva de Pistoia a partir de la tension (Voigt, Pa).

spinpy la calcula de sigma y eps; aqui eps sale de sigma por la flexibilidad isotropa, con distorsiones de ingenieria. Es la misma energia: $U = \sigma \cdot S \sigma / 2$, $eps_eff = \sqrt{2 U / E}$.

`huella(nodos, elems)`

SHA-256 de la malla (coordenadas float64 y conectividad int64, en C).

`caras_borde_tet10(elems)`

Caras de borde de una malla TET10: (elemento, tri6 saliente).

Una cara de esquinas que aparece en un solo tetraedro es de borde. Se orienta con la normal hacia FUERA del tetraedro (la esquina opuesta queda detras), que es lo que necesita la traccion en el techo, y se completa con sus nodos intermedios en el orden tri6 (esquinas a, b, c; aristas ab, bc, ca).

`opciones_malla(**kw)`

Opciones de la malla suave con sus valores por omision, validadas.

`maxvolume_de_tamano(L)`

Volumen del tetraedro regular de arista L: $L^3 / (6 \sqrt{2})$.

```
preparar_tet10(nodos, elems, forma, spacing)
```

Post-proceso de una malla TET10 dentro del cubo de la mascara.

El cubo es el de `solido.superficie_cerrada`: planos en $-h/2$ y $(n - 1/2)h$ en cada eje. Se exige que la MALLA una base y techo (se descartan los componentes que no toquen los dos planos z , y se reporta el volumen perdido), se extraen las caras de borde tri6 salientes, las del techo (cargadas) y la capa superficial: elementos con una cara de borde que NO este sobre un plano del cubo.

Devuelve (nodos, elems, vol, caras_techo, superficie, z0, z1, informe). Sirve igual para una malla que no salga de voxeles (la validacion con la esfera analitica usa esta misma funcion).

```
malla_de_tet10(nodos, elems, forma, spacing, informe=None)
```

Dict de malla (como el de `mallar`) a partir de un TET10 cualquiera.

```
mallar(BW, spacing, tipo='hex8', eje=2, progreso=None, **opciones)
```

Mascara -> malla lista para los motores, con el eje de carga en z .

Siempre se filtra PRIMERO a lo portante (`resistencia._solo_portante`, el mismo filtro del ensayo de la app): un fragmento que no une base y techo solo aporta una submatriz casi singular. Para cargar en X o Y se PERMUTA la mascara como `ensayo_compresion_eje` y se malla el resultado: la carga siempre va en z .

Devuelve un dict con tipo, nodos (N , 3) mm, elems (M , 8|10) base 0, caras_techo quad4 / tri6 cargadas, normal $+z$, vol_elem volumen de cada elemento (mm^3), superficie elementos de la capa superficial (bool, M): los que tienen una cara en el borde libre del hueso. Las caras sobre los seis planos del cubo NO cuentan (la regla de las isocaps, la misma de `resistencia.capa_superficie`). A_{bruta} , H , $z0$, $z1$, forma, spacing, eje, huella (SHA-256), informe.

Con TET10, `informe` declara la perdida de volumen frente a los voxeles del hueso portante y frente a la superficie cruda de marching cubes, y el volumen descartado al exigir que la MALLA (no la mascara) una base y techo: el suavizado puede cortar un puntal de un voxel.

```
tamano_previsto(BW, spacing, tipo, n=None)
```

Elementos, GDL y memoria previstos del resolovedor directo, sin mallar.

hex8: EXACTO (voxeles portantes y sus nodos a la resolucion n). tet10: `tiempos.n_tet10` a partir de los voxeles de superficie, $\pm 4\%$ en hueso trabecular (ver `tiempos`). Cuesta menos de un segundo a 48^3 .

```
memoria_equipo_MB()
```

RAM fisica total del equipo en MB (None si no se puede leer).

```
mallar_aislado(BW, spacing, tipo='tet10', eje=2, **opciones)
```

`mallar` en un PROCESO HIJO: si tetgen se cae, no se lleva la app.

tetgen es codigo C sin red: sobre el VOI proximal de H4 remuestreado a 20^3 (puntales de ~ 1 voxel, que el suavizado deja casi degenerados) mato el interprete con una violacion de segmento. En la GUI eso cerraba la ventana entera, con horas de sesion. Aqui el hijo muere solo y se lanza `ErrorMalla`, que la etapa registra como fallo y la cadena sigue. El hijo cuesta unos segundos de arranque (importar numpy y pyvista); con hex8 no hay tetgen y se malla en el mismo proceso.

```
protocolo(clave, **cambios)
```

Copia de un protocolo con `cambios` aplicados y su nombre de registro.

```
nombre_protocolo(clave, modificado=False)
```

`_Sin documentar._`

```
estadisticos(malla, sigma, prot, sigma_ref, F_total_Pa_mm2)
```

p99 de superficie y Pistoia de un campo de tensiones, ponderados por volumen.

Se usan las MISMAS funciones que la app (`criterio_pistoia`, `estadisticos_vm`), a las que se pasa `vol_solido`: con hexaedros de un voxel los pesos son iguales y el resultado es bit a bit el de siempre. Con TET10 la tension de cada elemento es la media de sus puntos de Gauss.

```
tension_elemental(nodos, elems, u, E, nu)
```

Tension (Voigt, Pa) media por elemento del campo lineal `u` (mm).

`hex8`: gradiente en el centro del voxel, la convencion de la app (`resistencia._b_centro`); con elementos rectangulares es la media de la deformacion sobre el elemento. `tet10` de aristas rectas: la deformacion es lineal en el elemento y su valor en el centroide es su media de volumen (lo mismo que escribia FEBio: la media de sus puntos de Gauss).

```
ensayo(malla, prot, carpeta=None, analisis=('lineal',), motor=MOTOR_DEF, hilos=None,
cancelar=None, material='svk', pasos=1, progreso=None, solver='auto', aislado=True,
BW=None, prefijo='ensayo', **_viejos)
```

Ensayo de compresion de un protocolo sobre `malla`, con `motor`.

Corre los `analisis` pedidos (ver `ANALISIS`) y devuelve un registro con procedencia. Un analisis que falla se registra y NO detiene los demas; solo `Cancelado` sale hacia arriba.

`lineal` el problema lineal a la carga del protocolo: `E_app`, fuerza (comprobacion de equilibrio), p99 de von Mises en la capa superficial y `Pistoia`. `nl_fuerza` no lineal a la carga del protocolo, traccion muerta, en `pasos` incrementos. `nl_plato` plato rigido sin friccion: lineal del plato y no lineal a la deformacion equivalente a la carga del protocolo (la del lineal con fuerza), como `comparativa_febio/plato.py`. `nl_pistoia` no lineal a la carga de fallo de `Pistoia` del lineal.

`E_app` se define con `uz` medio del techo PONDERADO POR AREA (`_uz_medio`); con `hex8` se da ademas `E_app_nodal` (media simple de nodos, la de la app). `BW` (la mascara de la malla `hex8`) solo lo necesita el motor '`app`'. `aislado`: cada resolucion en un proceso hijo cancelable (la GUI).

```
ensayo_app(BW, spacing, prot, eje=2)
```

El ensayo LINEAL de la app con los valores del protocolo.

Es la columna «`app` (`hex8`, `lineal`)» de la tabla. Mismo `resistencia` que el panel de Analisis mecanico; nada propio.

```
analizar(BW, spacing, prot, malla='hex8', analisis=('lineal',), eje=2, carpeta='.', n=None,
motores_fem=None, hilos=None, cancelar=None, material='svk', pasos=1, progreso=None,
comparar_app=True, opciones_malla=None, etiqueta='estructura', conv_malla=False,
solver='auto', aislado=True, motor=None, **_viejos)
```

Una estructura, un protocolo de compresion, una malla, un eje, y uno o VARIOS motores.

Es lo que corren la GUI («Informe FEM (Auto)...» y el ensayo del panel) y la CLI: remuestrea la mascara a `n` (vecino mas proximo, como la app; `None` = la resolucion de siempre de cada malla), malla UNA vez y resuelve la misma malla con cada motor de `motores_fem` (ensayo). Si `comparar_app`, corre ademas el ensayo lineal de la app sobre la MISMA mascara remuestreada a la resolucion de ladrillos (la columna «`app`»).

`conv_malla` (solo TET10): repite el lineal con la mitad del tamaño de elemento y sin decimar, con el primer motor, y guarda `convergencia_malla` (`dE_rel`, `dp99_rel`).

Devuelve UNA LISTA de registros, uno por motor, que comparten `app`, `estructura`, `n`, `tiempo_malla_s` y la malla (`_malla`). Los que no resuelven esa combinacion (p. ej. la app con TET10) se registran con el motivo en `fallos` y `no_disponible`.

```
tabla_motores(registros, referencia=None)
```

Filas de la TABLA COMPARATIVA ENTRE MOTORES.

Agrupar por (estructura, protocolo, malla, eje, n) y comparar cada motor con el de referencia (por omisión, el primero del grupo que resolvió el lineal): `dE_app`, `dp99` de superficie y `d sigma_fallo` relativos. Sobre la misma malla las diferencias miden solo la implementación; lo esperable es $\sim 1e-8$ o menos (la tolerancia de los resolvers iterativos).

```
registro_json(reg)
```

Copia serializable del registro: sin campos ni mallas en memoria.

```
fila_tabla(reg)
```

Números de la tabla comparativa de un registro (MPa y adimensionales).

`dE_app_implementacion` (solo hex8): el motor frente a la app con la DEFINICIÓN de la app (media simple de nodos del techo); debe ser ~ 0 . Las demás diferencias frente a la app mezclan malla y definición y se rotulan así en la GUI.

```
tensor_de_voigt(e)
```

Voigt [xx yy zz yz xz xy] con distorsiones de INGENIERÍA $\rightarrow$ tensor.

```
homogeneizar(BW, spacing, prot, malla='hex8', carpeta='.', n=None, progreso=None,
etiqueta='estructura', motores_fem=None, **_viejos)
```

Tensor elástico periódico sobre la malla de ladrillos.

Con hex8 es `elastic.homogeneizar`, el problema discreto que se valida contra FEBio a $1e-6$ del máximo de C (comparativa_febio/porcino/). Las cotas KUBC/SUBC sobre TET10 que daba la ruta de FEBio no están todavía en los motores internos: con tet10 se devuelve el registro con el motivo en fallos en lugar de un tensor.

```
main(argv=None)
```

```
python -m spinpy.fem VOI.vtk [...] --protocolo app --motores ngsolve
```

Clase `ErrorMalla`

El mallado TET10 fallo (tetgen abortado o proceso caído).

`_14` función(es) interna(s) de este módulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz pública._

Motores FEM: registro, detección y proceso hijo

```
__init__.py
```

`spinpy.motores` — Motores de elementos finitos que corren DENTRO de spinpy.

Sustituyen a FEBio (un ejecutable externo) en los ensayos de compresión con malla de ladrillos (hex8) o malla suave (TET10), lineales y no lineales. Todos reciben el MISMO problema (`problema_de_malla`) y devuelven el desplazamiento en el orden de nodos de spinpy, de modo que el postproceso (`E_app`, von Mises, Pistoia) es común y la comparación entre motores mide solo el motor.

app el resolver propio (`resistencia`); hex8 lineal. Siempre. ngsolve NGSolve (pip: ngsolve, mkl). hex8/TET10, lineal y no lineal. fenicsx FEniCSx (solo conda-forge; compila en tiempo de ejecución). skfem scikit-fem (pip). hex8/TET10, lineal y no lineal; lento en NL. sfepy SfePy (pip). hex8/TET10 lineal; no lineal solo SVK y con Newton de convergencia lineal (tangente inconsistente, medido).

La elección y las medidas que la justifican: `comparativa_motores/INFORME.md`.

`disponibles()` dice cuáles están instalados sin importarlos del todo; `resolver(problema, motor)` corre en el proceso actual y `resolver_aislado(...)` en un proceso hijo cancelable (lo usa la GUI: un fallo de memoria o de una biblioteca en C no cierra la aplicación, y «Detener» mata el cálculo en curso).

Funciones

`instalado(motor)`

`_Sin documentar._`

`disponibles()`

Motores instalados en este entorno, en el orden de `MOTORES`.

`version(motor)`

`_Sin documentar._`

`puede(motor, malla='hex8', analisis='lineal', material='svk', control='fuerza')`

True si `motor` resuelve esa combinacion (sin importarlo).

`resolver(problema, motor)`

Resuelve en este proceso. Devuelve `{u, F_reac, meta}`.

`rss_pico_MB()`

Pico de memoria residente de ESTE proceso, en MB (None si no se sabe).

Linux: VmHWM de `/proc/self/status`. NO `ru_maxrss`: en Linux ese maximo se conserva a traves de `exec`, y un hijo `spawn` heredaria el del proceso de la GUI en el momento del `fork` (medido: los tres motores de una prueba daban el mismo valor, el de la ventana). Windows: `PeakWorkingSetSize`.

`resolver_aislado(problema, motor, cancelar=None, hilos=None, intervalo=0.25, tiempo_max=None)`

`resolver` en un PROCESO HIJO (`spawn`), cancelable.

`cancelar`: funcion sin argumentos o `threading.Event`; si se activa, se termina el hijo y se lanza `Cancelado`. Un hijo que muere sin responder (memoria agotada, violacion de segmento en una biblioteca en C) se convierte en `ErrorMotor` en lugar de cerrar la aplicacion.

Clase `Cancelado`

El usuario detuvo el calculo; el proceso hijo ya se termino.

`_3` funcion(es) interna(s) de este modulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz publica._

Motores FEM: problema comun y resolvedores

`_comun.py`

`_comun.py`: Lo que comparten los adaptadores de motor.

EL PROBLEMA

Un motor recibe un dict `problema` (ver `problema_de_malla`) y devuelve `Solucion`: el desplazamiento nodal (N, 3) en el ORDEN DE NODOS DE SPINPY y la fuerza de reaccion en el techo por paso de carga. Unidades mm - N - MPa.

`nodos`, `elems` (N, 3) mm; (M, 8) hex8 C3D8 o (M, 10) TET10 C3D10 `caras_techo` quad4 / tri6 cargadas (normal +z) `caras_base` quad4 / tri6 de la base (esquinas primero) `base_nodos` todos los nodos del plano z0 (intermedios incluidos) `techo_nodos` todos los nodos del plano z1 `meta` tipo, apoyo, E, nu, H, A_bruta, `ancla_xy`, `ancla_y` y: `analisis` 'lineal' | 'nl' `control` 'fuerza': traccion p uniforme sobre las caras del techo (la tension aparente por la seccion BRUTA, repartida sobre el area osea, como la app); 'plato': uz = -eps H impuesto en todo el techo, sin friccion. `cargas` lista de p (MPa, control por fuerza) o de eps (plato), una por paso; el lineal usa la ultima. `material` 'svk' |

'neohookeano' (solo no lineal) solver 'directo' | 'iterativo' | 'auto' BW, spacing solo para el motor 'app' (su ensayo parte de la mascara)

La fuerza de reaccion se define igual en todos: $F = -\sum(f_{int}, z)$ sobre los GDL z del techo, con f_{int} el vector de fuerzas internas en el estado resuelto (positiva en compresion). En lineal con control por fuerza es la carga aplicada; con plato, la que hace el plato.

Funciones

`modos_rigidos(coord)`

Seis modos de solido rigido (3N, 6), GDL entrelazados x, y, z.

`modos_rigidos_gdl(xyz, comp)`

Modos rigidos para GDL con numeracion arbitraria.

`xyz` (n, 3): posicion del nodo de cada GDL; `comp` (n,): su componente. Tres traslaciones y tres giros infinitesimales: el espacio nulo de la rigidez sin apoyos, que el multigrad algebraico necesita representar en cada nivel grueso (ver el comentario del AMG en `resistencia.py`).

`resolver_scipy(A, b, B=None, modo='iterativo', tol=1e-10)`

Sistema disperso SDP con scipy/pyamg (motores sin resolvedor propio).

directo SuperLU (`scipy.sparse.linalg.splu`), ordenacion COLAMD. iterativo CG preconditionado con multigrad algebraico de agregacion suavizada (pyamg) y los modos rigidos `B` como espacio nulo cercano: la receta de `resistencia.ensayo_compresion`. Devuelve (x, info) con el residuo relativo $\|b - Ax\| / \|b\|$; si no alcanza 100 tol se lanza `ErrorMotor` (un iterativo no avisa solo).

`emparejar(coord_motor, coord_spinpy, tol_rel=1e-09)`

Indice `i` tal que `coord_motor[i[k]] == coord_spinpy[k]`.

Los motores reenumeran nodos y GDL; se vuelve al orden de spinpy por coordenadas. Si un nodo no tiene pareja se aborta: la solucion no se podria atribuir a la malla de spinpy.

`esquinas(elems)`

Nodos de esquina y conectividad reenumerada sobre ellos.

`lame(E, nu)`

(lambda, mu) de Lamé a partir de E y nu.

`rectificar_aristas(nodos, elems)`

TET10 con cada nodo intermedio en el punto medio de su arista.

`fem.preparar_tet10` devuelve a su plano exacto los nodos a menos de $1e-3$ h de un plano del cubo; si una esquina se mueve y la otra no, el nodo intermedio deja de estar en el punto medio (medido: hasta $8e-4$ h). Los motores construyen P2 sobre las esquinas, con aristas RECTAS: para que todos resuelvan la misma geometria se recoloca cada intermedio. Devuelve (nodos, desplazamiento maximo).

`problema_de_malla(malla, E, nu, apoyo='deslizante', analisis='lineal', control='fuerza', cargas=(1.0,), material='svk', solver='auto', tol=1e-10, BW=None)`

Problema de motor a partir de una malla de `fem.mallar`.

E en MPa, cargas en MPa (control por fuerza: tension aparente sobre la seccion BRUTA; aqui se convierte en la presion sobre el area osea del techo) o adimensionales (plato: deformacion aparente).

`fijos_basicos(p)`

[(nodo, componente)] del apoyo (sin el plato).

```
solucion(u, F, meta, **extra)
```

Salida comun de un motor.

Clase `Cronometro`

Tiempos por etapa (s, reloj de pared).

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`, `fin`

Clase `NoDisponible`

El motor no resuelve esta combinacion (elemento, analisis, material).

Clase `ErrorMotor`

El motor fallo o no convergio.

Motor `NGSolve`

`m_ngsolve.py`

`m_ngsolve.py`: Motor NGSolve (nucleo C++ con interfaz Python; LGPL-2.1).

La malla de spinpy pasa a netgen sin transformarla (`AddPoints`, `AddElements`): hexaedros de un voxel, o tetraedros por sus esquinas con espacio P2, y las caras de base y techo como elementos de borde con nombre. Las formas son simbolicas:

lineal $\sigma(u) : \epsilon(v) \, dx = -p \, v_z \, ds(\text{techo})$ no lineal $\text{Variation}(W(F) \, dx - p \, u_z \, ds(\text{techo}))$: NGSolve deriva la energia y la linealiza (`AssembleLinearization`); Newton propio con predictor consistente del plato.

Resolvedores: directo = PARDISO de MKL (paquete `mk1` de pip; sin el, `sparsecholesky` de NGSolve). Iterativo: con P2, CG + BDDC (el preconditionador de subestructuracion de NGSolve); con hex8 (orden 1) BDDC degenera en un directo, asi que se usa CG + pyamg con modos rigidos sobre la matriz que monta NGSolve.

Funciones

```
version()
```

`_Sin documentar._`

```
resolver(p)
```

`_Sin documentar._`

`_11` funcion(es) interna(s) de este modulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz publica._

Motor `FEniCSx`

`m_fenicsx.py`

`m_fenicsx.py`: Motor FEniCSx (DOLFINx + UFL + FFCx + PETSc; LGPL-3.0).

No esta en PyPI: se instala con conda-forge (`conda install -c conda-forge fenics-dolfinx`). Las formas se escriben en UFL; FFCx las traduce a C y CFFI las COMPILA la primera vez (hace falta un compilador de C en tiempo de ejecucion; quedan en cache en `~/cache/fenics`). Ese tiempo se mide aparte ("jit").

lineal $a(u, v) = \sigma(u) : \epsilon(v) \, dx$, $L(v) = -p \, v_z \, ds(\text{techo})$ no lineal $Pi(u) = W(F) \, dx + p \, u_z \, ds(\text{techo})$; residuo = `derivative(Pi)`, tangente derivada por UFL; Newton de PETSc (SNES newtonls).

Resolvedores de PETSc: directo = Cholesky de MUMPS; iterativo = CG + GAMG con los seis modos rigidos como espacio nulo cercano.

Funciones

`version()`

`_Sin documentar._`

`resolver(p)`

`_Sin documentar._`

`_4` funcion(es) interna(s) de este modulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz publica._

Motor scikit-fem

`m_skfem.py`

`m_skfem.py`: Motor scikit-fem (Python puro sobre numpy/scipy; BSD-3).

scikit-fem monta formas bilineales y lineales escritas en Python y devuelve matrices de scipy; no trae resolovedor ni Newton. Aqui: lineal `linear_elasticity` + traccion en el techo; SuperLU o CG + pyamg con modos rigidos (`resolver_scipy`). no lineal formas de Lagrange total escritas a mano (St. Venant-Kirchhoff y neo-Hookeano de FEBio) y Newton-Raphson propio con predictor consistente del plato. Elementos: `hex8` = `ElementHex1` (Gauss 2x2x2); `tet10` = `ElementTetP2` sobre las esquinas (aristas rectas).

Funciones

`version()`

`_Sin documentar._`

`mm(A, B)`

Producto matricial de campos tensoriales (3, 3, elementos, puntos).

`resolver(p)`

`_Sin documentar._`

`_3` funcion(es) interna(s) de este modulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz publica._

Motor SfePy

`m_sfepy.py`

`m_sfepy.py`: Motor SfePy (Python con extensiones C/Cython; BSD-3).

Usa los terminos de SfePy (`dw_lin_elastic`, `dw_surface_ltr`, `dw_tl_he_svk`), su Newton y su capa de resolovedores: directo `ls.scipy_direct` (SuperLU; UMFPACK no esta en pip); iterativo `ls.scipy_iterative` (CG) con el preconditionador pyamg con modos rigidos inyectado por `setup_precond`, el mecanismo de SfePy para eso (`ls.pyamg` no admite el espacio nulo cercano).

No lineal: solo St. Venant-Kirchhoff. MEDIDO (comparativa_motores/): la tangente de `dw_tl_he_svk` en SfePy 2026.3 difiere un 37 % de la derivada numerica de su propio residuo, asi que Newton converge LINEALMENTE (22-27 iteraciones por paso en un bloque; no converge en 30 al 2 % en un espinodoide). El residuo es correcto: cuando converge, la fuerza coincide con la solucion cerrada. El neo-Hookeano de SfePy es la variante desacoplada ($\mu/2 (J^{2/3} I_1 - 3) + K/2 (J - 1)^2$), que no es el de FEBio: no disponible.

Funciones

`version()`

Sin documentar.

`resolver(p)`

Sin documentar.

4 funcion(es) interna(s) de este modulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz publica.

Motor de la app

`m_app.py`

`m_app.py`: El resolvidor propio de spinpy (`resistencia.ensayo_compresion`).

Es el motor de siempre y el validado: hexaedros de un voxel, elasticidad lineal, control por fuerza, LU directo por debajo de 6000 GDL y CG con multigrid algebraico (pyamg, modos rigidos) por encima. No tiene elementos TET10, ni plato rigido, ni analisis no lineal: esas combinaciones lanzan `NoDisponible`.

Funciones

`version()`

Sin documentar.

`resolver(p)`

Sin documentar.

Mallas solidas: hexaedrica y TET10

`solido.py`

`solido.py` — Exportacion del spinodoide (o de un VOI) como SOLIDO.

Un STL es solo la piel: describe la frontera, no el interior. Para simular hace falta una malla VOLUMETRICA. Este modulo produce las dos variantes que tienen sentido sobre una mascara de voxeles.

HEXAEDRICA (directa desde los voxeles)

Cada voxel solido se convierte en un hexaedro de 8 nodos. Es exacta —reproduce la mascara sin ninguna aproximacion—, instantanea y NO PUEDE FALLAR: no hay mallador que se atragante. Es el metodo estandar de micro-elementos finitos para hueso trabecular (Van Rietbergen, Ruegsegger) y es lo que hacen `localWriteFEBioHex` y `localWriteAbaqusHex` de la app de MATLAB.

Su limitacion: el hexaedro trilineal es un elemento LINEAL, mas rigido a flexion que uno cuadratico con los mismos grados de libertad, y la superficie queda escalonada.

TETRAEDRICA TET10 (cuadratica)

Superficie por marching cubes, suavizada, reparada y tetraedralizada. La superficie sale lisa y el elemento cuadratico representa mejor la flexion, a cambio de ser mucho mas lento y de introducir tres pasos donde se pierde algo de material.

TRES COSAS QUE COSTARON DESCUBRIR Y QUE NO SE PUEDEN OMITIR

- 1 **SUAVIZAR ES OBLIGATORIO** antes de tetraedralizar. Sobre la superficie cruda de marching cubes tetgen aborta con `recoversubfaces` o `split_segment`: los datos binarios producen miles de triangulos coplanares y aristas diminutas que no puede recuperar. No es cosmetica.
- 1 **EL ORDEN DE NODOS DE TETGEN NO ES EL DE C3D10**. Medido (distancias exactamente cero):
ranura tetgen 4 5 6 7 8 9 arista (2,3) (0,3) (0,1) (1,2) (1,3) (0,2) C3D10 espera (0,1) (1,2) (0,2) (0,3) (1,3) (2,3)
De ahí `PERM_TETGEN_A_C3D10`. Sin la permutacion los elementos quedan retorcidos y el analisis corre igualmente devolviendo tensiones sin sentido: un error que no da ningun sintoma. Por eso se **VERIFICA** sobre la malla concreta antes de escribir.
- 1 **SISTEMA DE UNIDADES mm-N-MPa**. Ni Abaqus ni ANSYS llevan unidades: interpretan los numeros en el sistema coherente que elija quien escribe. La malla esta en milimetros, asi que el modulo se escribe en MPa. Poner $20e9$ (pascals) declara un material 10^6 veces mas rigido y el analisis corre sin protestar.

Funciones

`malla_hex(BW, spacing)`

Un hexaedro por voxel solido. Devuelve (nodos, elems, informe).

Solo se crean los nodos que algun elemento usa: una mascara con BV/TV de 0.3 generaria de otro modo un 70% de nodos sueltos que ningun elemento referencia, y varios preprocesadores los rechazan.

`volumen_hex(nodos, elems)`

Volumen total, por el jacobiano en el centro de cada hexaedro.

Sirve de comprobacion independiente: si la conectividad estuviera en un orden equivocado los volúmenes saldrian negativos.

`superficie_cerrada(BW, spacing)`

Isosuperficie 0.5 cerrada por relleno de vacio.

El relleno no es solo para cerrar: como en el corte el gradiente es puramente normal, las caras de los bordes salen PLANAS en vez de abiertas, que es lo que se necesita si despues hay que apoyar la probeta.

`malla_tet10(BW, spacing, suavizado=SUAVIZADO_ITER, banda=SUAVIZADO_BANDA, decimado=DECIMADO, progreso=None, maxvolume=None, minratio=None, ejes_planos=(2, ), volumen_objetivo=None)`

Mascara -> TET10. Devuelve (nodos, elems, superficie, informe).

Los cuatro ultimos argumentos son opcionales; con su valor por omision el resultado es el de siempre.

`maxvolume` volumen maximo de tetraedro (mm^3) para tetgen; `None` deja que el tamaño lo marque la superficie. `minratio` razon radio-arista maxima de la mejora de calidad de tetgen (su defecto es 2.0). `ejes_planos` tapas que se devuelven a su plano exacto tras el suavizado. Por omision solo z (base y techo del ensayo). (0, 1, 2) deja las seis caras del cubo planas: lo necesitan las condiciones de borde de la homogeneizacion y la regla que excluye las caras del cubo de la capa superficial. `volumen_objetivo` si se da (mm^3), la superficie reparada se desplaza por su normal para encerrar ese volumen (dos iteraciones) antes de tetraedralizar.

`verificar_tet10(nodos, elems, tol_rel=1e-06, n_muestra=2000)`

Comprueba que cada nodo intermedio esta en el medio de SU arista.

`_6` funcion(es) interna(s) de este modulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz publica._

Exportadores: Abaqus, APDL, VTU, STL

`escribe.py`

`escribe.py` — Escritores de malla para Abaqus, ANSYS, ParaView y STL.

SISTEMA DE UNIDADES: mm - N - MPa

Ni Abaqus ni ANSYS llevan unidades; interpretan los numeros en el sistema coherente que elija quien escribe el modelo. Las mallas de este proyecto estan en MILIMETROS, porque el spacing del micro-CT lo esta, y el sistema coherente con el milimetro es:

longitud mm fuerza N tension y modulo MPa = N/mm²

Las funciones reciben E en PASCALES —como lo maneja el resto del proyecto, `localBaseMaterial` da 20e9— y lo escriben en MPa. Volcar 20e9 tal cual declara un material un millon de veces mas rigido, y el analisis corre sin protestar: solo se nota al ver desplazamientos absurdamente pequenos.

TIPOS DE ELEMENTO

hex8 Abaqus C3D8 ANSYS SOLID185 tet10 Abaqus C3D10 ANSYS SOLID187

Funciones

`escribir_stl(superficie, ruta)`

Sin documentar.

`escribir_vtu(nodos, elems, ruta, arrays=True)`

Malla volumetrica en VTU, para inspeccionarla en ParaView o pyvista.

Es el formato util para MIRAR la malla antes de mandarla a un solver: los .inp y .apdl son enormes y ningun visor los abre comodamente.

POR QUE LLEVA ARRAYS. Una malla sin ningun campo escalar es geometria muda: ParaView no puede colorearla ni construir una tabla de color, y al pedirle la representacion "Volume" responde

Failed to determine the LookupTable being used. Could not determine array range.

que suena a archivo corrupto y solo significa que no hay nada que mapear. Se incluyen por tanto la calidad y el volumen de cada elemento, que ademas son lo que de verdad interesa mirar: permiten localizar en pantalla los elementos degenerados antes de mandar la malla a un solver.

La calidad se evalua sobre la celda LINEAL —el hexaedro, o el tetraedro de las cuatro esquinas en el caso del TET10— porque el filtro de calidad de VTK no soporta celdas cuadraticas y devuelve -1 para todas, lo que aparenta una malla catastrofica cuando en realidad es que no sabe medirla. Como los TET10 son de lados rectos, la calidad de forma es la del tetraedro de esquinas y la medida es exacta, no una aproximacion.

`escribir_abaqus(nodos, elems, ruta, E_s=200000000000.0, nu_s=0.3, nombre='SOLIDO', conjuntos=True)`

Malla volumetrica en formato Abaqus .inp.

ANSYS Mechanical lo importa directamente (External Model), asi que el mismo archivo sirve para las dos plataformas. Se escriben los conjuntos de nodos BASE y TECHO para que aplicar el apoyo y la carga en el preprocesador sea un clic y no una seleccion manual sobre cientos de miles de nodos.

`escribir_febio(nodos, elems, ruta, E_s=200000000000.0, nu_s=0.3, sigma_app=1000000.0, A_bruta=None, apoyo='deslizante', datos=True, eps_plato=None)`

Ensayo de compresion en z listo para correr en FEBio 4 (`febio4 -i`).

No es solo la malla: reproduce el MISMO problema que resuelve `resistencia.ensayo_compresion`, para que el resultado de FEBio pueda enfrentarse numero a numero con el de spinpy.

material 'isotropic elastic' de FEBio (St. Venant-Kirchhoff): se reduce a la elasticidad lineal de spinpy para deformaciones pequenas. La diferencia es de orden de la deformacion del tejido; con cargas pequenas es despreciable y, como el problema es lineal, se puede cargar poco y reescalar. carga presion NO seguidora (Linear = 1) sobre las caras superiores de los elementos que tocan el techo. La fuerza total es $\sigma_{app} * A_{bruta}$, repartida sobre el area osea del techo: misma convencion de seccion BRUTA que spinpy. FEBio integra la presion por su cuenta, asi que tambien pone a prueba el reparto por area tributaria de spinpy (un cuarto por nodo y cara). apoyo 'deslizante': $u_z = 0$ en la base, (u_x, u_y) en la esquina (min x, min y) y u_y en la (max x, min y), las mismas esquinas geometricas que elige spinpy. 'empotrado': base fija entera. solver un paso, Newton completo (max_ups = 0), Pardiso directo: independiente del AMG + CG de spinpy.

`A_bruta` es la seccion del VOI en mm^2 . Si no se da, se toma la caja de los nodos, que COINCIDE con la del VOI salvo que el filtro de portantes haya vaciado una columna del borde: en ese caso hay que pasarla.

Con `datos` se piden al logfile los desplazamientos nodales (<ruta>_u.txt) y las tensiones de Cauchy por elemento en orden Voigt del proyecto, xx yy zz yz xz xy (<ruta>_s.txt). FEBio da la media de los ocho puntos de Gauss, que en un hexaedro rectangular es exactamente el valor en el centro que usa spinpy. Las reacciones NO se piden: FEBio 4.5 las escribe como cero en los GDL de un `zero displacement` (comprobado tambien con la variable del plotfile). El equilibrio se comprueba con la integral de volumen: $\sum(\sigma_{zz} V_e) = -F H$, exacta en el problema discreto.

`eps_plato` cambia el control de carga: en lugar de la presion, un PLATO RIGIDO sin friccion impone el mismo $u_z = -\epsilon_{plato} * H$ a todos los nodos del techo (u_x, u_y libres). No es el ensayo de la app —que controla la fuerza— sino su contrapunto: con fuerza impuesta, una trabecula cortada por la cara del VOI recibe carga en su extremo libre y trabaja en voladizo; con el plato, ese extremo se mueve con el resto del techo. `sigma_app` se ignora en ese caso; la fuerza se mide despues, por la misma integral de volumen de la tension.

Unidades mm - N - MPa, como el resto de escritores: E y sigma se reciben en Pa y se escriben en MPa.

`area_caras(nodos, caras)`

Area de cada cara quad4 (rectangulo de voxel) o tri6 (lados rectos).

```
escribir_febio_ensayo(nodos, elems, ruta, E_s=20000000000.0, nu_s=0.3, sigma_app=1000000.0,
A_bruta=None, apoyo='deslizante', datos=True, eps_plato=None, caras_techo=None,
material='svk', pasos=1, comentario='', adaptativo=False)
```

El ensayo de `escribir_febio`, generalizado a TET10 y a varios pasos.

Hermano de `escribir_febio`, que NO se toca: es el camino hex8 validado contra la app a $\leq 1e-6$ en `comparativa_febio/`. Con hexaedros, un paso y St. Venant-Kirchhoff, este escribe el MISMO problema linea a linea salvo el comentario de cabecera (lo comprueba el bloque 27), asi que esa validacion se hereda.

Lo que anade: tet10 elementos cuadraticos en orden C3D10 —el que deja `solido.malla_tet10`; que FEBio use el mismo se MIDE en el bloque 27 con un campo cuadratico exacto, no se supone— y carga sobre caras `tri6` del techo. `caras_techo` caras cargadas (quad4 o tri6), con la normal hacia +z. FEBio aplica la presion contra la normal de la cara: una cara del reves tira del techo en vez de comprimirlo, sin ningun error. Con hexaedros y None se toman como siempre. material 'svk' (St. Venant-Kirchhoff, el validado) o 'neohookeano'. pasos incrementos de carga del analisis no lineal. adaptativo escribe un `time_stepper` que RECORTA el paso si Newton no converge (siempre que pasos > 1). Medido en el VOI proximal de H4 a $20^{\wedge}3$: con 1, 5 y 10 pasos la respuesta no lineal es la misma a todas las cifras y el coste crece con los pasos (5, 20 y 40 iteraciones), asi que el defecto es un paso con recorte automatico.

Apoyo, presion y plato son los de `escribir_febio`. Con TET10 la base y el techo son planos EXACTOS (`solido._aplanar_tapas`), asi que base y techo son todos los nodos de esos planos, intermedios incluidos, y las anclas del apoyo deslizante salen de la misma regla (min x+y, max x-y): en una malla suave no hay esquina geometrica, y esa regla elige el nodo de la base mas extremo en esas direcciones.

```
escribir_apdl(nodos, elems, ruta, E_s=20000000000.0, nu_s=0.3)
```

Script de Mechanical APDL con NBLOCK/EBLOCK y conjuntos de nodos.

3 funcion(es) interna(s) de este modulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz publica.

Lote de VOIs y dispersion del generador

lote.py

lote.py — Ajuste y generacion de replicas en lote, con salida tabular.

PARA QUE SIRVE Y PARA QUE NO

El objetivo del proyecto es aumentar el N en estudios de volúmenes óseos escasos. Conviene ser preciso sobre que hace y que no hace este modulo, porque la diferencia decide si un resultado es publicable:

GENERAR K REPLICAS DE UN VOI NO DA $N = K$. Da $N = 1$ con K replicas tecnicas. La varianza total se descompone en la de ENTRE especimenes —la biologica, que fijan los animales— y la de DENTRO, que es el ruido del generador. Las replicas dividen la segunda por raiz de K y NO TOCAN la primera. Para una afirmacion sobre "el hueso sesamoideo equino" los grados de libertad los ponen los caballos, no las realizaciones.

Lo que las replicas SI compran:

- reducir la incertidumbre de la estimacion DE CADA especimen
- experimentos numericos controlados que el hueso real no permite (variar BV/TV con DA fijo, por ejemplo)
- rellenar huecos del muestreo
- alimentar un modelo sustituto estructura -> propiedad

Por eso la tabla que produce este modulo lleva SIEMPRE las columnas animal, sitio y replica separadas: sin esa estructura no se puede descomponer la varianza despues, y el error solo se detecta cuando ya se gastaron semanas de CPU. spinpy.estadistica opera directamente sobre ellas.

REPRODUCIBILIDAD

Cada fila registra la semilla y los parametros exactos que la produjeron, de modo que cualquier espécimen sintético se puede regenerar bit a bit.

Funciones

```
generar_desde_parametros(par, semilla)
```

Mascara de un ajuste a partir de su dict parametros, de cualquier familia.

La traduccion en si vive en procedencia.kwargs_generador, que es la UNICA del paquete y comprueba que las dos lecturas del numero de onda cuadran: un parametro con el valor del deslizador tomado por radianes falla ahi en vez de generar una estructura pi veces mas gruesa.

```
dispersion_semillas(parametros, spacing, n_semillas=8, semilla_base=None, m_ref=None, extra=False, progreso=None)
```

Repite la MISMA parametrizacion con K semillas y resume la dispersion.

PARA QUE SIRVE. El generador es estocastico: dos evaluaciones del mismo vector de parametros dan realizaciones distintas. Esa dispersion es el SUELO DE RUIDO por debajo del cual ninguna diferencia significa nada — ni entre dos ajustes, ni entre Python y MATLAB, ni entre un candidato y el VOI. Toda la validacion del port se apoya en ella, y hasta ahora habia que cambiar la semilla a mano y anotar los numeros.

Si se pasa `m_ref` (típicamente la morfometría del VOI), cada métrica trae además su `z`: cuantas desviaciones típicas del propio generador separan al VOI de la media de las realizaciones. Es el mismo criterio con el que se valida el port — $|z| < 2$ es indistinguible del ruido— y convierte una diferencia porcentual, que no dice nada por sí sola, en una afirmación comprobable.

Un CV pequeño NO significa que la métrica sea fiable: significa que el generador la reproduce de forma estable. BS tiene un CV del 0.19% a densidad 0.55 y aun así arrastra el sesgo de marching cubes.

`cerrar_obreros()`

Cierra los procesos obreros que `loky` deja vivos para reutilizarlos.

`joblib` mantiene sus obreros arrancados unos minutos después de que `Parallel` devuelva, para no pagar el arranque la próxima vez. El precio es que HEREDAN LA SALIDA ESTÁNDAR del proceso padre: si alguien canaliza la salida del programa, la tubería no ve nunca el final aunque el padre haya terminado. Compilando se perdieron 27 minutos por esto, con un obrero girando al 100 % de una CPU después de que el guión hubiera acabado.

Se llama al terminar el lote. Si el backend no es `loky` no hay nada que cerrar y no pasa nada.

`opciones_paralelo(n_jobs=-1)`

Como paralelizar, según se corra desde el código o desde el ejecutable.

DESDE EL CÓDIGO: procesos. Es lo correcto — el trabajo es CPU pura y el GIL lo serializaría.

EMPAQUETADO CON PyInstaller: hilos, y no es una preferencia sino la única opción que funciona. `joblib` usa `loky`, que arranca a sus obreros reejecutando `sys.executable` con argumentos del estilo `-c "from loky ..."`. En un ejecutable congelado `sys.executable` es la propia aplicación, que no acepta `-c`: los obreros mueren nada más nacer y `joblib` levanta `TerminatedWorkerError`. Medido sobre el .exe construido aquí: el obrero muere a los 24 s y el proceso padre se queda colgado.

Con hilos se pierde parte del paralelismo, pero no todo: generar la máscara, la morfometría y la homogeneización pasan casi todo su tiempo dentro de `numpy`, `scipy` y `pyamg`, que sueltan el GIL. El lote tarda más en el ejecutable que desde el código, y conviene saberlo antes de lanzarlo.

```
correr_lote(vois, n_replicas=10, modo='completo', mecanica=False, res_homog=16,
E_s=20000000000.0, nu_s=0.3, semilla_base=770000, n_jobs=-1, progreso=None,
familia='spinodoide', opciones_ajuste=None)
```

Ajusta cada VOI y genera `n_replicas` sintéticas de cada ajuste.

familia: "spinodoide" (por omisión, el comportamiento de siempre) o "dual-lattice". Las dos se ajustan con la misma búsqueda escalonada y el mismo modo; la tabla lleva una columna `familia` para que dos lotes de familias distintas se puedan concatenar sin mezclarse.

opciones_ajuste: dict que se pasa tal cual al ajuste (`pesos`, `distancia`, `replicas`, `seleccion`, `k_robusto`, `alineacion...`). Lo que se uso queda en la tabla de ajustes, columna `objetivo`.

PROCEDENCIA. Cada fila de las dos tablas lleva las columnas de `procedencia.COLUMNAS` (versión de `spinpy` y del formato, familia, esquema, número de onda con sus dos lecturas). En las filas sintéticas `semilla` es la de ESA replica y la del ajuste va en `semilla_ajuste`.

Devuelve un `DataFrame` ordenado, una fila por espécimen:

origen 'real' para el VOI medido, 'sintético' para las réplicas animal H1..H4, C1..V5 — la unidad de inferencia biológica sitio proximal / medio / distal replica 0 para el real, 1..K para las sintéticas semilla la que generó esa realización (NaN en los reales) ... parámetros ajustados y métricas

La fila real de cada VOI se incluye con `replica = 0` para que la comparación real-vs-sintético salga de la misma tabla y no de dos ficheros que puedan desincronizarse.

3 funcion(es) interna(s) de este modulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz publica.

Varianza, ICC, N efectivo y equivalencia TOST

estadistica.py

estadistica.py — Descomposicion de varianza y equivalencia real vs sintetico.

EL ERROR QUE ESTE MODULO EXISTE PARA IMPEDIR

Generar K replicas de cada VOI y despues tratarlas como K especimenes independientes es pseudorreplicacion (Hurlbert 1984). Infla los grados de libertad, estrecha artificialmente los intervalos de confianza y convierte cualquier diferencia en "significativa".

La varianza total se descompone en

$\sigma^2_{\text{total}} = \sigma^2_{\text{entre}} + \sigma^2_{\text{dentro}}$

donde `entre` es la variabilidad biologica entre especimenes y `dentro` la del generador. Promediar K replicas divide σ^2_{dentro} por K y deja σ^2_{entre} INTACTA. El N efectivo para una inferencia biologica lo fijan los especimenes, no las realizaciones.

`n_efectivo` lo calcula explicitamente para que aparezca en cualquier informe y sea dificil confundirlo con el numero de filas de la tabla.

POR QUE TOST Y NO UNA t DE STUDENT

Para sostener que los sinteticos SUSTITUYEN a los reales hace falta demostrar EQUIVALENCIA, y "no se rechaza la hipotesis nula" no es eso: con N pequeno no se rechaza nunca y con N grande se rechaza siempre, asi que la prueba clasica responde a la pregunta contraria a la que interesa.

TOST invierte la carga: se declara de antemano un margen y se demuestra que la diferencia cae dentro. Y ese margen no hay que inventarlo — el suelo de ruido del propio generador, ya medido en `Validacion_Anexo`, es el limite fisico por debajo del cual dos realizaciones de los MISMOS parametros tampoco se distinguen. Pedir mas seria pedir que el sintetico se parezca al real mas de lo que el real se parece a si mismo.

Funciones

```
descomponer_varianza(df, metrica, col_grupo='animal', col_replica='replica', solo=None)
```

Componentes de varianza entre y dentro de espécimen, ICC y N efectivo.

Modelo de efectos aleatorios de una via sobre los especimenes, con el estimador clasico de ANOVA (Searle):

$\sigma^2_{\text{dentro}} = \text{CM}_{\text{dentro}}$ $\sigma^2_{\text{entre}} = (\text{CM}_{\text{entre}} - \text{CM}_{\text{dentro}}) / n_0$

con n_0 el tamano medio armonico de grupo. σ^2_{entre} puede salir NEGATIVA cuando la variabilidad entre especimenes es menor que el ruido; se acota a cero y se marca, porque un valor negativo no es interpretable pero tampoco es un fallo del calculo.

```
descomponer_anidada(df, metrica, col_animal='animal', col_sitio='sitio',  
col_replica='replica', solo=None)
```

Descomposicion ANIDADA: animal / sitio(animal) / replica.

POR QUE NO BASTA UN SOLO NIVEL. Agrupar por `animal` mete en el mismo saco la variacion entre SITIOS del mismo animal y el ruido del generador, que son cosas distintas: la primera es biologica y la segunda es numerica. Sobre los VOIs equinos —4 animales x 3 sitios x K replicas— esa mezcla inflaba el CV "dentro" hasta duplicar el "entre" y hacia el ICC ininterpretable.

El diseno real tiene tres niveles y el modelo tiene que tener tres:

$$\sigma^2_{\text{total}} = \sigma^2_{\text{animal}} + \sigma^2_{\text{sitio}(\text{animal})} + \sigma^2_{\text{replica}}$$

ANOVA anidada clasica (Sokal & Rohlf) sobre un diseno balanceado. Los componentes pueden salir negativos cuando un nivel aporta menos que el ruido del siguiente; se acotan a cero y se marca, porque un valor negativo no es interpretable pero tampoco es un fallo de calculo.

El N para inferir entre animales sigue siendo el numero de ANIMALES.

```
tost(x, y, margen_rel, alfa=0.05)
```

Prueba de equivalencia por dos t unilaterales (Schuirmann).

Concluye equivalencia si el intervalo de confianza $(1-2*\alpha)$ de la diferencia de medias cae ENTERO dentro de $\pm$ -margen. `margen_rel` es relativo a la media de `x` (el grupo de referencia), que es como se declara de forma natural a partir de un coeficiente de variacion.

```
informe(df, metricas, margen_rel=None, col_grupo='animal')
```

Descompone la varianza y contrasta equivalencia real vs sintetico.

`margen_rel` por metrica; si falta, se usa el CV DENTRO observado, que es el suelo de ruido del propio generador y por tanto el margen mas exigente que tiene sentido pedir.

Linea de comandos

```
cli.py
```

`cli.py` — Entrada por linea de comandos, sin rutas propias de nadie.

POR QUE EXISTE. `correr_lote.py` sirve para trabajar, pero lleva el banco de VOIs escrito dentro:

```
BANCO = Path(r"C:\Users\carlo\OneDrive\Escritorio\VOIs Caballos")
```

Eso esta bien para un script personal y es exactamente lo que impide que otra persona use la herramienta: la primera ejecucion falla en una ruta que no existe en su maquina, y el mensaje no le dice que la cambie. Aqui las rutas son argumentos obligatorios.

```
spinpy-lote VOI_*.vtk --replicas 20 --salida ./resultados
```

El script antiguo se conserva: tiene los valores por defecto del banco de este proyecto y ahorra teclear. Los dos llaman al mismo `correr_lote`.

Funciones

```
main_lote(argv=None)
```

```
_Sin documentar._
```

Interfaz grafica

```
visor.py
```

`visor.py` — Interfaz grafica del port: VOI real y spinodoide candidato, lado a lado.

ALCANCE — menor que el de `AppFinal_V2`, y solo lo que esta validado

Esta ventana expone lo que el nucleo portado respalda: generacion del spinodoide, carga de un VOI, morfometria comparada, ajuste por busqueda escalonada, tensor elastico homogeneizado, ensayo de compresion en uno o en los tres ejes con campos de deformacion efectiva y de von Mises, y exportacion como SOLIDO (malla hexaedrica o TET10) para Abaqus y ANSYS.

NO hay informes ni los otros cuatro optimizadores (Pareto, bayesiano, MOBO, best-fit rapido): esa logica sigue viviendo solo en MATLAB. Un boton apagado seria peor que un boton ausente.

SESION REPRODUCIBLE

El menu Sesion guarda en JSON todo lo que determina el estado: parametros, semilla, esquema de muestreo, resoluciones y la ruta del VOI. Como el generador es estocastico pero esta sembrado, ese archivo regenera la MISMA realizacion bit a bit. Sin el, un caso interesante que se pierde al mover un deslizador no se recupera.

EXPORTAR: HEXAEDRICA O TET10

La hexaedrica pone un elemento por voxel. Es EXACTA —reproduce el 100% del volumen—, instantanea y no puede fallar al mallar, pero el elemento es lineal y la superficie queda escalonada.

La TET10 suaviza la superficie y la tetraedraliza: elemento cuadratico y piel lisa, a cambio de tiempo y de perder algo de material. Cuanto se pierde depende de la RESOLUCION: medido sobre el mismo spinodoide, 88.8% del volumen a 40^3 y 93.8% a 48^3 . Con trabeculas de uno o dos voxeles la superficie no tiene con que trabajar, asi que conviene exportar a la resolucion mas alta que se pueda pagar.

ORIENTACION IMPUESTA POR EL AJUSTE (correccion F4)

Los deslizadores de rotacion solo cubren ± 90 grados y no pueden representar todas las orientaciones de un VOI. Cuando el ajuste deriva una R del eje principal, esa R manda sobre los deslizadores y se avisa en el panel; mover cualquier rotacion la libera, porque ese gesto expresa que el usuario quiere controlar la orientacion a mano.

DOS SUPERFICIES DE MEDIDA, Y HAY QUE DECLARAR CUAL

La seccion Morfometria tiene un desplegable "Superficie":

- voxeles — marching cubes crudo sobre la mascara. Es el modo de la app de MATLAB, el que usa el ajuste, y el que sobreestima el area un ~ 8.5 % (medido sobre una esfera). Como VOI y candidato lo sufren igual, el sesgo se cancela al comparar.
- malla — superficie cerrada, suavizada con Taubin y reparada con PyMeshFix, recortada medio voxel por dentro y sin las seis tapas (`spinpy.morphometry.morfometria_malla`). Quita ese sesgo: sobre el VOI proximal de H4, BS pasa de 412 a 377 mm² (-8.7 %) y Tb.Th sube un 4.7 %. BV/TV sale $\sim 1-3$ % menor, que es la diferencia entre la superficie a nivel 0.5 y el conteo de cubos enteros, no una perdida: la perdida real de suavizar y reparar se mide (-0.4 % en ese VOI) y se ensena en la barra de estado.

Cambiar de modo vuelve a medir el VOI: una tabla con el VOI en un modo y el candidato en otro mezcla dos definiciones de BS (correccion C4, otra vez). El modo viaja en la sesion y en `_res["morfometria"]["modo"]`, para que cualquier tabla exportada diga con que superficie se midio.

EL PANEL VA POR ETAPAS DEL TRABAJO

El panel izquierdo esta dividido en secciones con cabecera propia, en el orden en que se usan: VOI de referencia, Spinodoide, Visualizacion, Morfometria, Ajuste al VOI, Analisis mecanico, Exportacion, y Lote y varianza. Antes todo colgaba del mismo nivel: los grupos existian, pero no se veia de un vistazo en que etapa estaba cada control ni donde correspondia anadir uno nuevo.

Cada seccion se abre con `_seccion`, que dibuja la raya separadora, el titulo y la raya bajo el titulo, y devuelve el layout donde va el contenido. Anadir una seccion es una linea; anadir un control es meterlo en el layout que devuelve.

DOS RESOLUCIONES, QUE ES LO QUE HACE USABLE LA INTERFAZ

Generar y medir a 64^3 cuesta ~ 9 s en esta maquina, lo que vuelve inservible un deslizador. Aqui se separan:

- Resolucion de VISTA — se recalcula al mover un deslizador. A 32^3 con pocas ondas baja a decimas de segundo y sirve para explorar la forma.
- Resolucion de MEDIDA — solo se usa al pulsar "Medir". Es la que produce numeros comparables con el VOI, y es la que hay que citar.

Ambas usan la MISMA semilla, asi que muestrean el mismo campo aleatorio a distinta finura: la vista previa es una version gruesa de lo que se medira, no otra estructura.

El esquema de muestreo de ondas es un desplegable y no tiene valor por defecto silencioso: 'rechazo' (GIBBON/AppFinal_V2) y 'equitativo' (TPMS-Scaffolds-generator) producen anisotropias distintas para los mismos thetas cuando los conos tienen angulos desiguales. Ver README.

DOS IDIOMAS: ESPANOL E INGLES

El menu **Idioma** cambia toda la interfaz sin reconstruir la ventana ni perder nada: el VOI cargado, las metricas medidas y los campos de deformacion siguen donde estaban. Son minutos u horas de calculo y no se pueden tirar por cambiar de idioma.

COMO. `spinpy.idioma.capturar()` recorre el arbol de widgets UNA vez al arrancar y anota el texto ORIGINAL de cada uno; `aplicar()` los reescribe traducidos. Como el original queda guardado, ir y volver entre idiomas es idempotente. Lo que ese recorrido no ve -mensajes de la barra de estado, cuadros de dialogo, informes en HTML- lleva `_()` explicito donde se construye.

Los textos cuyo valor cambia en marcha -la etiqueta de un deslizador dice "Densidad relativa: 35 %" - se marcan con la propiedad `i18n_dinamico` para que el recorrido los salte, y se retraducen ellos solos.

EL DICCIONARIO VA INDEXADO POR LA CADENA ESPANOLA (`spinpy/idioma_textos.py`). Eso mantiene el codigo legible y hace que un texto sin traducir salga en espanol en vez de romperse, pero tiene un filo: cambiar una tilde en un texto espanol y no tocar el diccionario pierde la traduccion inglesa SIN NINGUN ERROR. Para eso esta `idioma_revisar.py`, que construye la ventana de verdad, recorre sus widgets, saca del codigo las cadenas envueltas en `_()` y lista lo que falta, lo que sobra y los marcadores `{ }` que no cuadran entre las dos versiones. Hay que ejecutarlo despues de tocar cualquier texto:

```
python idioma_revisar.py (resumen; devuelve 1 si falta algo) python idioma_revisar.py --esqueleto (deja las claves listas para pegar)
```

QUE NO SE TRADUCE, A PROPOSITO:

- los nombres de metrica -BV/TV, Tb.Th, Tb.Sp, Tb.N, BS/BV, DA, SMI, Conn.D-, que son notacion internacional (Parfitt / Bouxsein et al. 2010);
- las unidades y los simbolos (mm, MPa, GPa, sigma, epsilon, E_{app});
- las citas bibliograficas;
- el informe de `--autocomprobacion`, que es una herramienta de diagnostico para enviar por correo, no parte de la interfaz.

La eleccion se guarda en `QSettings("spinpy", "visor")` y se restaura al abrir.

LA FABRICA SE DIBUJA, NO SOLO SE TABULA

La casilla "Mostrar fabrica MIL" pinta en cada panel el elipsoide del tensor MIL en alambre y, en magenta, su eje mayor: la direccion en la que la estructura es mas continua y por tanto mas rigida. En el panel del spinodoide se rotula ademas el angulo entre ese eje y el del VOI.

POR QUE HACIA FALTA. El tensor MIL se calcula desde el principio, pero de el solo se ensenaban dos escalares: DA y DA2. Dos estructuras pueden tener el MISMO DA y estar orientadas a 90 grados una de otra, y la tabla no las distingue. En el ajuste del VOI proximal de H4 el candidato ganador resulto tener su eje rigido a 87 grados del hueso, con lo que su modulo aparente caia de 633 a 11 MPa; eso se supo despues de 477 s de ajuste y de leer el dialogo del analisis comparado. Dibujado, se ve antes de ajustar nada.

El rotulo pasa a rojo si el angulo supera 30 grados, y avisa aparte cuando $DA2 < 1.06$: por debajo de ese umbral la fabrica es un PLANO y no un eje, la flecha apunta a una direccion cualquiera de ese plano, y ninguna rotacion convierte esa estructura en axial. Es el mismo criterio con el que la correccion L1 (`spinpy/fit.py`) decide abstenerse de alinear.

CONVENCION, que es donde se equivoca uno. El ajuste del tensor es $1/MIL(n)^2 = n' M n$, de modo que el MIL en la direccion del autovector i vale $1/\sqrt{\lambda_i}$: el autovalor MENOR corresponde al MIL MAYOR. El elipsoide se dibuja con semiejes proporcionales a $1/\sqrt{\lambda}$, no a λ , que lo pintaria aplastado justo en la direccion en la que la estructura es continua. El semieje mayor se lleva al 62 % del lado y las flechas al 85 %, para que asomen por las caras: inscritos quedarian enterrados dentro de una estructura opaca y no se veria nada.

VIGENCIA. El elipsoide del spinodoide describe la ULTIMA MEDIDA, no la posicion de los deslizadores. Al regenerar la vista desaparece hasta que se vuelva a medir, en vez de quedarse pegado: una flecha de la estructura anterior sobre la geometria nueva es indistinguible a la vista de un resultado valido.

RENDER: SOMBRAS DE CONTACTO Y FONDO

Una trabecula es una marana de barras cruzadas. Sin sombra donde una pasa por detras de otra, la profundidad se lee solo por el escorzo y el conjunto parece plano. La casilla "Sombras de contacto (SSAO)" activa oclusion ambiental de pantalla; el radio NO es fijo, sino una fraccion de la diagonal de la caja, porque en milímetros absolutos un VOI de 5 mm y un cubo unidad de 1 mm darian resultados opuestos. Se anade tambien suavizado SSAA -las aristas de la isosuperficie salian dentadas en las capturas- y un punto de especularidad en el material.

Todo esto es APARIENCIA: no toca ninguna medida, y las tres casillas se pueden desmarcar si el driver de video lo dibuja mal o si el equipo va lento.

El fondo por defecto es un degradado suave, que da a la silueta algo contra lo que recortarse. "Fondo blanco (para figuras)" devuelve el blanco puro, que es el correcto para una figura de tesis.

Uso: `python visor.py`

Funciones

```
mejor_ajuste(VOI, spacing, registro, claves, m_voi=None, semilla=20260720, num_waves=700,
informar=None)
```

El ajuste de MENOR ERROR entre los metodos `claves` de `registro`.

Una sola definicion de «el mejor ajuste» para los dos sitios que la usan: el analisis comparado con «Ajustar antes» y el informe automatico. Si cada uno eligiera a su manera, el informe podria describir un ganador distinto del que el analisis comparado ensayo con el mismo VOI.

Solo tiene sentido con metodos que comparten la definicion de error (`METODOS_COMPARABLES`): el desempate mecanico suma dos terminos y su cifra no se puede poner en la misma cola.

Corre en el hilo que llama, en serie. `informar(i, n, etapa)` recibe el avance sobre el total de evaluaciones de todos los metodos. Devuelve (`ganador, hechos`): el resultado ganador, con `_todos` resumiendo el error y el tiempo de cada metodo, y el dict `metodo -> resultado entero`.

```
opciones_figuras()
```

Estilos, vistas y suavizado de las figuras 3D del informe.

Se guardan en QSettings al aceptar la ventana del informe automatico y los lee tambien el informe manual. Lo guardado se valida contra `figuras.opciones_3d`: una clave vieja o un estilo que ya no exista no rompen el informe, toman el valor por omision.

```
guardar_opciones_figuras(o)
```

Sin documentar.

```
factores_tiempo()
```

({etapa: real/estimado}, numero de ejecuciones) de ESTE equipo.

`spinpy.tiempos` esta medido en el equipo de desarrollo; cada etapa que termina en el informe automatico corrige su factor (`guardar_factor_ tiempo`), asi que la estimacion se acerca a este equipo con el uso.

```
guardar_factor_tiempo(clave, real, estimado)
```

Corrige el factor de `clave` con una etapa terminada.

Se descartan las etapas de menos de 20 s: ahi manda el coste fijo y el cociente es ruido. `estimado` ya incluia el factor anterior, asi que el nuevo factor es el anterior por el cociente de esta ejecucion.

```
apoyo_de_combo(cmb)
```

Clave CANONICA del apoyo ("deslizante"/"empotrado") de un combo de apoyo.

Se lee por INDICE, nunca por texto: con la interfaz en ingles el texto es "fixed"/"sliding", y `ensayo_compresion` no lo entiende (antes lo tomaba en silencio por deslizante; ahora lanza ValueError). Lo que se pasa al ensayo y lo que se guarda en los registros es siempre esta clave; para mostrarla en pantalla, `_(clave)`.

```
mallade_mascara(BW, spacing)
```

Isosuperficie 0.5 de la mascara como PolyData.

Se construye un ImageData con la mascara en los NODOS y se contornea: es marching cubes de VTK, el mismo algoritmo que mide `area_superficie`, de modo que lo que se ve es lo que se mide.

`flatten(order='F')` porque VTK recorre X como indice mas rapido y BW tiene dimension 1 = X - la misma convencion que `readVTKV0I`.

Las normales se calculan EXPLICITAMENTE: `contour()` devuelve la malla sin ellas y, sin normales, `smooth_shading` no tiene con que sombrear. La estructura se renderiza entonces como una silueta plana de un solo color y parece un bloque macizo aunque tenga BV/TV de 0.28.

```
autocomprobacion()
```

Ejercita sin interfaz cada camino de calculo y escribe un informe.

PARA QUE SIRVE La aplicacion empaquetada arranca aunque le falte una biblioteca: los modulos pesados -tetgen, pymeshfix, pyamg- no se importan hasta que el usuario pulsa el boton correspondiente. Un fallo de empaquetado en el mallador no se manifiesta al abrir la ventana, sino media hora despues, en mitad de una exportacion, en forma de traza que quien la recibe no sabe leer. Esto lo destapa en unos segundos y en la maquina donde importa: la del usuario, no la del que compilo.

El informe se guarda ademas en un archivo, para poder enviarlo.

Uso: `spinpy.exe --autocomprobacion` (o `python visor.py --auto...`) Devuelve 0 si todo pasa, 1 si algo falla.

```
main()
```

Sin documentar.

Clase Trabajador

Ejecuta una funcion lenta fuera del hilo de la interfaz.

Sin esto la ventana se congela durante la generacion y Windows la marca como 'no responde', que es justo lo que ocurre en la app de MATLAB con `drawnow` cuando la resolucion sube.

```
Trabajador.informar(self, i, n, etapa)
```

Callback de progreso para tareas largas.

Se invoca DESDE EL HILO DE TRABAJO. Emitir una senal es la forma correcta de cruzar al hilo de la interfaz: Qt la encola y la entrega en el bucle de eventos. Tocar los widgets directamente desde aqui provoca caidas intermitentes muy dificiles de reproducir.

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`, `run`

Clase `DialogoPilaTiff`

Carga de una pila TIFF de micro-CT, con su escala fisica.

POR QUE UN DIALOGO Y NO UN `getOpenFileName` A SECAS. Un .vtk trae dentro el tamano de voxel; una pila de imagenes no trae NADA. Si se abriera sin preguntar habria que suponer una escala, y una escala equivocada multiplica en silencio Tb.Th, Tb.Sp, TV, Conn.D y la rigidez aparente sin producir un solo sintoma visible: las figuras salen igual de bonitas y todos los numeros estan mal. Por eso la escala es obligatoria y se ensena de donde sale.

Lo mismo con el umbral. Medido sobre H4, mover la segmentacion un +-15 % mueve Tb.Th un 62 %: es la mayor fuente de incertidumbre de todo el proceso, asi que se declara cual se aplica en vez de esconderlo.

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`, `valores`

Clase `DialogoRecorte`

Recorte del VOI cubico sobre el volumen cargado, con vista previa.

POR QUE ESTE PASO NECESITA INTERFAZ. Una pila de escaner no es un VOI: 628x664x438 son 1.8e8 voxeles, y ni la homogeneizacion ni el ajuste son viables ahi. El recorte ya existia (`vois_por_tercios`, `extraer_cubo`) pero solo desde un script, que es tanto como decir que no existia para quien no programa.

SOBRE LOS NOMBRES ANATOMICOS. El signo de PC1 es arbitrario: la PCA determina las direcciones, no su sentido, y dos ejecuciones sobre datos casi iguales pueden devolver ejes opuestos. Por eso `vois_por_tercios` devuelve los tercios ORDENADOS a lo largo del eje y sin nombre, y por eso aqui la asignacion proximal/distal la hace quien conoce la pieza, con el BV/TV como pista: en estos sesamoideos la densidad crece de proximal a distal. Poner el nombre por nuestra cuenta seria acertar la mitad de las veces y no avisar de la otra mitad.

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`

Clase `DialogoFigura`

Base de los dialogos que muestran una figura con su lectura al pie.

Todos comparten la misma estructura —lienzo de matplotlib, tabla o nota en HTML debajo, y un boton para guardar la figura— y la misma regla: **ninguna figura se muestra sin el texto que dice como leerla**. Un histograma de espesor sin la advertencia del sesgo del 30%, o una curva de convergencia sin el veredicto, invitan a citar un numero que no es citable.

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`

Clase `DialogoEspesor`

Distribucion del espesor local, VOI y spinodoide superpuestos.

POR QUE UN HISTOGRAMA Y NO SOLO Tb.Th. El Tb.Th de la tabla es un escalar derivado de Parfitt (2*Bv/BS): supone un modelo de placas y colapsa toda la estructura en un numero. Dos VOIs con el mismo Tb.Th pueden tener uno las trabeculas homogeneas y el otro una mezcla de trabeculas gruesas y finas —que es justo la firma

de la perdida osea— y el escalar no los distingue. La distribucion si.

QUE NO SE PUEDE HACER CON ESTOS NUMEROS. La escala absoluta NO es citable como Tb.Th: a 3-4 voxels de grosor el metodo de esferas inscritas subestima un ~30%. Sirve para comparar dos estructuras medidas con el MISMO spacing y para leer la FORMA de la distribucion (moda, cola, bimodalidad). Los dos paneles comparten eje x por esa razon; con ejes distintos la superposicion no significaria nada.

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`

Clase `DialogoConvergencia`

E_app frente al tamaño de elemento, con el veredicto al pie.

La app ya advertia de que E_app no converge sin mas en estructuras poco densas, pero no ofrecia forma de comprobarlo. Esta figura es esa forma.

SE TRAZAN DOS COSAS EN EL MISMO EJE. Arriba E_app, que es lo que se quiere; abajo la **fraccion portante**, que es el diagnostico. Si la fraccion portante sube al refinar, la estructura conectada esta cambiando, y entonces la variacion de E_app no es error de discretizacion: son geometrias distintas. Sin esa segunda curva, una serie que sube y baja parece ruido numerico y se descarta por el motivo equivocado.

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`

Clase `DialogoDistribucionFE`

Distribucion de la deformacion efectiva y de la tension de von Mises.

POR QUE LA DISTRIBUCION Y NO EL MAXIMO. El criterio de Pistoia no mira el maximo —seria un solo elemento, y en una malla de voxels ese elemento suele ser una esquina escalonada sin significado fisico—: mira el **percentil 98**, el valor que supera el 2% del tejido. Eso es un punto de la cola, y para saber si es robusto hay que ver la cola entera. Si esta concentrada en unos pocos nudos, el criterio depende de esos nudos; si esta repartida, el resultado es estable.

Se traza en escala logaritmica en x: los dos campos abarcan varios ordenes de magnitud entre el material apenas cargado y los nudos, y en escala lineal todo se amontona contra el cero.

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`

Clase `DialogoDispersion`

Suelo de ruido del generador: K realizaciones de los mismos parametros.

LA PREGUNTA QUE CONTESTA. Cuando el candidato difiere del VOI en un 3% en Tb.Sp, ¿es que el ajuste falla o es que el generador no reproduce Tb.Sp mejor que eso? Sin esta medida no hay forma de saberlo, y la diferencia porcentual sola invita a interpretar ruido como senal.

La columna **z** es el criterio con el que se valido el port: cuantas desviaciones tipicas del propio generador separan al VOI de la media de las realizaciones. $|z| < 2$ significa indistinguible del ruido, y no se puede pedir a un ajuste que acierte mas de lo que el generador se repite a si mismo.

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`

Clase `DialogoVarianza`

Descomposicion de varianza del lote, con el N efectivo en primer plano.

EL ERROR QUE ESTE DIALOGO EXISTE PARA IMPEDIR. Generar K replicas de cada VOI y tratarlas despues como K especimenes independientes es pseudorreplicacion: infla los grados de libertad, estrecha los intervalos de confianza y convierte cualquier diferencia en significativa. El numero de filas de la tabla NO es el N del estudio.

Por eso el N efectivo va arriba y en grande, y no escondido en una columna.

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`

Clase `DialogoSimulacion`

Resultado de una simulacion in silico: curvas, tabla y el 3D de cada paso.

Es el unico dialogo de figura con visor 3D, y no por adorno: una curva de rigidez que cae en picado no dice POR QUE. Ver que en ese paso desaparecio la ultima trabecula vertical, o que el dano se concentro en un cuello, es lo que convierte la curva en una explicacion.

La tabla lleva el residuo y la fraccion portante de cada paso por la misma razon que `DialogoConvergencia`: un E que cae porque la estructura se desconecto y un E que cae porque el solver no convergio no son el mismo resultado, y solo esas dos columnas los separan.

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`, `done`

Clase `Deslizador`

Deslizador con etiqueta y valor, en unidades reales (no pasos enteros).

ALTURA MINIMA FIJA. Sin ella, cuando el panel no cabe a lo alto Qt reparte el deficit comprimiendo cada hijo, y la etiqueta acaba solapada sobre la barra: ilegible. Con un minimo, el panel prefiere desbordar —y entonces la `QScrollArea` que lo envuelve muestra una barra de desplazamiento— antes que aplastar los controles.

`Deslizador.retraducir(self)`

Reescribe la etiqueta SIN emitir `cambiado`.

La senal esta conectada a `_pedir_vista` y a `_soltar_R`: emitirla al cambiar de idioma relanzaria el calculo de la vista y, peor, liberaria la orientacion que impuso el ajuste. Cambiar de idioma no puede tener efectos secundarios.

`Deslizador.fijar(self, v)`

Coloca un valor, acotandolo al rango del control.

Se acota en vez de fallar porque una sesion guardada con otra version puede traer un valor fuera de los limites de ahora; quedarse en el extremo y seguir es preferible a no cargar nada. Qt no emite `valueChanged` si el valor no cambia, asi que el aviso al usuario no puede colgar de la senal.

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`, `valor`

Clase `DialogoBienvenida`

Portada: que es esto, quien lo firma y por donde se entra.

Se abre ANTES de construir la ventana principal, no despues. Construir el `Visor` entero -con su render de VTK- tarda unos segundos, y durante ese rato la pantalla se queda vacia; asi la primera respuesta es inmediata.

Las características que se enumeran son las que la aplicacion tiene DE VERDAD y estan verificadas: esto es la portada de una herramienta de medida, no un folleto. El enlace al repositorio se dibuja a partir de `informe.REPOSITORIO`; mientras siga siendo `[PENDIENTE]` sale como texto apagado, y el dia que se rellene esa constante se convierte en enlace solo, sin tocar esta clase. Los autores salen de `informe.AUTORES`, que `tests/test_19` cuadra con `CITATION.cff`, para que la portada y la cita del software no puedan discrepar.

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`, `showEvent`, `closeEvent`, `mostrar_al_arrancar`

Clase `DialogoInformeAuto`

Que etapas corre el informe automatico, cuanto tardara y que figuras.

Las resoluciones y casillas NO son copias propias: salen de los controles del panel y, al aceptar, se escriben de vuelta en ellos. Asi las etapas leen los mismos controles que al pulsar sus botones, y la sesion guardada o exportada declara

los valores con los que se calculo.

TIEMPO ESTIMADO. Cada etapa lleva al lado lo que se espera que tarde, y el total se rehace con cada cambio (`spinpy.tiempos`). El modelo esta medido en el equipo de desarrollo y se corrige con lo que tardaron las etapas en ESTE equipo en ejecuciones anteriores; la nota de abajo dice cuantas hay.

La carpeta se pide AQUI, al principio, y no al final: la cadena puede tardar horas y tiene que poder terminar sin nadie delante.

```
DialogoInformeAuto.plan(self)
```

Lo que se va a correr, en los terminos de `spinpy.tiempos`.

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`, `familias`, `opciones`

Clase Visor

```
Visor.abrir_bienvenida(self)
```

La portada, ya con la ventana abierta y en el idioma en curso.

```
Visor.abrir_validacion(self, clave=None)
```

Abre la ventana de replicas, en la pestana `clave` si se pide.

El dialogo vive en `dialogo_validacion.py` y cada replica en su `Test/replicar_*.py`, que se pueden correr sin interfaz. Aqui solo se abre.

```
Visor.cambiar_idioma(self, cod, guardar=True)
```

Reescribe la interfaz en `cod` SIN reconstruirla ni perder estado.

Reconstruir la ventana seria mas simple de programar y peor de usar: se perderian el VOI cargado, las metricas medidas y los campos de deformacion, que son minutos u horas de calculo. En vez de eso se vuelven a escribir los textos sobre los mismos widgets.

Lo que hay que refrescar a mano es lo que NO es un widget: los rotulos dibujados dentro de los paneles 3D y las celdas de la tabla, que se escriben en cada medida y no en la construccion.

```
Visor.cargar_pila_tiff(self)
```

Carga un volumen directamente desde las imagenes del micro-CT.

Entra por el mismo `_instalar_voi` que un `.vtk`: en cuanto hay mascara y spacing en mm, todo lo de aguas abajo —morfometria, ajuste, homogeneizacion, von Mises, exportacion— funciona sin saber de donde vino.

```
Visor.recortar_voi(self)
```

Orienta el volumen cargado por PCA y abre el recorte del cubo.

```
Visor.medir(self, sobre_mascara=False)
```

Morfometria de cada familia activa y, si hace falta, del VOI.

`sobre_mascara=True` mide la mascara que ya tiene cada familia en vez de regenerarla a la resolucion de medida. Es lo que usa el informe automatico despues de ajustar: los deslizadores redondean densidad, numero de onda y angulos a su paso, asi que regenerar desde ellos mediria una estructura vecina de la ganadora, no la ganadora, y su huella no cuadraria con la del paquete de reproduccion.

```
Visor.ajustar_todos(self)
```

Ajusta con todos los metodos, compara y deja elegir.

```
Visor.motor_fe(self)
```

Clave del motor elegido en el panel ('app' si no hay combo).

`Visor.ensayo_fe(self)`

Ensayo de compresion sobre cada familia activa, una tras otra.

`Visor.analisis_comparado(self)`

Analisis comparado del VOI frente a cada familia activa.

`Visor.exportar_solido(self)`

Exporta cada familia activa, una detras de otra, a su archivo.

`Visor.calcular_elastico(self)`

Tensor elastico de cada familia activa, una tras otra.

`Visor.dispersion(self)`

Dispersion entre semillas de cada familia activa, una tras otra.

`Visor.exportar_resultados(self)`

Vuelca a CSV y JSON todo lo calculado en la sesion.

DOS ARCHIVOS, PORQUE SIRVEN PARA COSAS DISTINTAS. El CSV es plano y largo (una fila por magnitud) para pegarlo en una hoja o leerlo con pandas; el JSON conserva lo que no cabe en una tabla —el tensor C entero, los parametros, la traza— y es el que permite reproducir.

Se exporta lo que YA se calculo, sin recalculer nada. Con un generador estocastico, recalculer al exportar daria numeros distintos de los que el usuario vio en pantalla, y el archivo dejaria de ser el registro de lo ocurrido.

`Visor.informe_publicacion(self)`

Metodos en ES/EN, citabilidad de cada resultado y reproduccion.

Describe lo que YA se calculo, igual que la exportacion. Lo unico que ejecuta es el generador, para anotar la huella de cada estructura ajustada, y por eso va en un hilo de trabajo.

`Visor.informe_auto(self)`

Todo el recorrido sobre el VOI cargado y, al final, el informe.

Encadena los MISMOS metodos que los botones del panel, una etapa detras de otra, en vez de tener un camino de calculo propio: un informe automatico que midiera distinto que el manual describiria una sesion que nadie puede repetir a mano. Lo unico que cambia es que los dialogos de resultados no se abren y los errores se anotan.

`Visor.analizar_febio(self)`

«Comparar motores...»: la estructura activa y el VOI, con varios motores FEM sobre la misma malla.

`Visor.fem_auto(self)`

«Informe FEM (Auto)...»: todo desatendido, como el informe automatico, encadenando las mismas `_febio_una` que el panel.

Metodos sin docstring propio (su comportamiento se explica en la clase): `__init__`, `histograma_espesor`, `cargar_voi`, `ajustar`, `malla_fe`, `analisis_fe`, `distribuciones_fe`, `convergencia_fe`, `simular_perdida_gui`, `fallo_progresivo_gui`, `guardar_sesion`, `cargar_sesion`, `correr_lote_gui`, `closeEvent`

`_10` funcion(es) interna(s) de este modulo no se listan; su nombre empieza por guion bajo y no forman parte de la interfaz publica._